Seventeen years later: stateful neural guidance and the tail that sizes a Mars aerocapture mission

Grégory Gelly

gregory.gelly@gmail.com

Preprint, 2026

Abstract

In 2009 we showed that a single-hidden-layer feed-forward network, trained by a genetic algorithm, could fly the aerocapture of a Mars Sample Return vehicle more efficiently than a Cerimele–Gamble feedback law, and we closed that work by asking for the obvious next step: a comparison against predictor–corrector guidance. This paper answers it. We train stateful neural guidance policies and benchmark them, on identical Monte Carlo scenarios drawn from a bit-validated simulator with an independent density-noise realization per scenario, against six classical schemes retuned on the same objective, including a numerical predictor–corrector (FNPAG) and a reference-tracking feedback law (FTC). Because the mission's correction propellant is sized off the worst-case Δv, we lead every comparison with the tail of its distribution, not the mean. On 10^6 pre-registered confirmatory scenarios a 962-parameter recurrent (Mamba) policy trained from scratch holds a far-tail $\mathrm{CVaR}_{99.9}$ of 158.5 ± 15.8 m/s over three training seeds, 19 m/s below the best classical scheme (a well-referenced FTC, 177.8) and 31 below FNPAG, at 3.3 ms per simulation, $63\times$ faster than FNPAG; the deployed seed captures every scenario and exceeds the heat-load ceiling on 0.11% of them, where the classical law exceeds nothing. The seed spread is the finding's limit: a 515-parameter dense network reaches 160.5 ± 23.2 on the same pool, so the networks beat the classical schemes by a margin the spread does not erase, and beat one another by a margin it does. The result rests on a training methodology that is itself a contribution: a non-stationary, adaptive-seed Monte Carlo environment turns the genetic algorithm from the **worst** optimizer under fixed scenarios into the **best**, and a policy trained on one density history fits that history – the shared-path champions of our development regime lose 2–4 $\times$ more tail than the classical laws when each scenario gets its own realization. The deployed policy computes with its state (zeroing it every tick multiplies its CVaR_{95} by more than 6), but matched controls without learned state or without the engineered cost-to-go inputs reach the same tail within seed variance. The main deployment caveat: under a deliberately harsher off-nominal regime the analytic law generalizes better than the network, a gap that a regime-matched training objective closes at capture parity, at the price of more frequent heat-flux exceedance.

1 Introduction

Aerocapture replaces a propulsive orbit-insertion burn with a single pass through a planet's atmosphere: the vehicle enters on a hyperbolic trajectory and bleeds its excess energy through aerodynamic drag, modulating only the bank angle to steer the lift vector and arrive at a bound target orbit. The maneuver is attractive precisely because it is propellant-free, but it is also unforgiving. The control authority is proportional to dynamic pressure, the entry corridor is narrow, and the dispersions – atmospheric density first among them – are large. There is no closed-form optimal bank-angle law, so the classical practice is to make strong simplifying assumptions (a decoupling of in-plane and out-of-plane motion, a reference trajectory built on apoapsis control alone) in order to obtain a tractable feedback or predictor–corrector law. These simplifications are exactly what costs correction propellant downstream.

Neural networks are especially valuable for guidance problems that admit no analytic solution, and aerocapture is a textbook case. In 2009 we showed that a feed-forward network with a single hidden layer, trained by a genetic algorithm without any reference trajectory or input–output pairs, could perform the aerocapture of a Mars Sample Return (MSR) vehicle at a mean correction cost of 116.7 m/s against 144.8 m/s for a Cerimele–Gamble-derived feedback law – a 19% reduction, approaching the 113 m/s periapsis-raise floor (Gelly and Vernis, 2009), itself building on earlier aerocapture-corridor and soft-landing guidance studies (Vernis *et al.*, 2004; Gelly and Ferreira, 2007). The mechanism was that the network learned the in-plane and out-of-plane logic **jointly**, where the classical law treats them separately. We closed that paper with an explicit hook: "the next step would be to extend our work on the aerocapture to skip entry missions and evaluate the performance of neural guidance compared to classic algorithms such as the predictor-corrector schemes." This paper is that next step, seventeen years later.

In the interval we carried the same evolutionary-training philosophy to recurrent networks for speech, where a coordinated-gate LSTM trained by quantum-behaved particle-swarm optimization (Sun, Feng and Xu, 2004), with a divide-and-conquer initialization and task-aligned differentiable losses, became the working method (Gelly and Gauvain, 2015; Gelly and Gauvain, 2017; 2016). Stateful cells, swarm training, smart initialization, and custom losses are precisely the machinery this paper brings back to where it started. With their innate ability to exploit the trajectory history, recurrent and selective state-space policies are natural candidates for a guidance law that must anticipate the atmospheric pass rather than merely react to the current state.

We make three contributions:

1. **A training methodology, not just a policy.** We show that a genetic algorithm is the **worst** optimizer for this problem under a fixed set of Monte Carlo scenarios – it overfits the repeated cases – and the **best** under a non-stationary, adaptive-seed environment that keeps moving the scenarios beneath it. Switching the seed schedule from fixed to moving takes the mean correction cost from 154 to 120 m/s in three-seed means (first seeds $160.3 \rightarrow 118.0$), the single largest effect in the campaign. The moving environment, a tail-weighting cost transform, and hardest-case seed curation form one matched system, and the noise regime the policy trains on is part of the match: a network trained on one shared density history fits it, and only training under per-scenario noise produces a policy that scores the same under both.
2. **An architecture finding, with its limit measured.** Across dense, gated-recurrent (GRU, LSTM), windowed, attention (Transformer), and selective state-space (Mamba) cells, the engineered, cost-aligned inputs flatten the **median** correction cost to a common 110–116 m/s for every architecture trained to convergence. At the far-tail depth where the tanks are sized, every network family's seed mean sits below the best classical scheme (19 m/s for the Mamba's three seeds), but the families sit inside one another's seed spread, and three matched controls – state reset, an explicit observation window, input removal – show that the deployed policy uses its state without showing that it needs it. Training loss does not pick the tail winner: the deployed seed has the lowest validation loss of its family and the worst far tail.
3. **A systematic head-to-head of neural versus predictor–corrector aerocapture guidance** – to our knowledge the first for an MSR-class Mars aerocapture to compare an end-to-end learned policy against classical baselines co-tuned on the same objective, on paired dispersed scenarios, under a far-tail correction-Δv risk metric. On identical dispersions the deployed network beats the best classical scheme by 15.9 m/s in mean and 26.6 m/s at CVaR_{95}, winning 99.4% of paired scenarios, at a per-simulation compute cost 63 $\times$ below the numerical predictor–corrector – with the caveat that the analytic law is more robust off-nominal, under a deliberately harsh stress regime we return to in Section 7.3.

This answer lands in a field that has moved since 2009. On the classical side, FNPAG (Lu *et al.*, 2015) set the numerical predictor–corrector standard and was carried across Mars aerocapture mission and vehicle design maps (Matz *et al.*, 2017); more recent work replans under uncertainty explicitly – two-stage stochastic and robust formulations that beat a deterministic predictor–corrector under density perturbations (Zucchelli *et al.*, 2021) – solves the full constrained replan by convexification (Rataczak, McMahon and Boyd, 2025), or augments the bank channel with angle-of-attack modulation through optimal control (Sonandres, Palazzo and How, 2025a). Machine learning has so far entered the loop mostly as a **component** inside a classical scheme: LSTM density estimation feeding FNPAG (Sonandres, Palazzo and How, 2025b), generative failure-mode indicators steering a predictor–corrector (Calkins *et al.*, 2025). End-to-end learned aerocapture guidance evaluated against classical baselines on common dispersed

scenarios – with the classical side given the same objective and tuning freedom, and the comparison read at the far-tail depth that sizes the propellant – is, to our knowledge, the gap this paper fills.

All of this rests on a high-fidelity simulator – $J_2/J_3/J_4$ gravity, Gauss–Markov density perturbations, thermal limits, pilot dynamics – regression-validated against a legacy reference implementation across all 725 time steps of a guided trajectory, 22 of 24 output channels bit-identical (the two mismatches trace to uninitialized variables in the reference; Appendix A lists the independent physics checks and the simulator features this campaign does not exercise). The next section formalizes the aerocapture problem and the sizing-tail objective; Section 3 describes the guidance schemes; Section 4 presents the training methodology; Sections 5–7 give the optimizer, architecture, and classical-versus-neural results; and Section 8 reports what the deployed network actually uses.

2 Problem and objective

2.1 Dynamics and the aerocapture corridor

We study the same robotic MSR mission concept as the 2009 work. The vehicle reaches the atmospheric entry interface at an altitude of 130 km with a relative velocity of 5687 m/s, a flight-path angle of $-10.81°$, and an azimuth of $38.04°$ (the 2009 study placed the interface at 120 km with a $-10.24°$ flight-path angle; vehicle and target are unchanged); it carries a mass of 1089 kg on a 14.7 m² reference area and is statically trimmed at a fixed angle of attack, so the only control is the bank angle μ, slewed at up to 15°/s. The guidance targets a 500×11 km orbit at atmosphere exit (apoapsis $\times$ periapsis altitude), at 50° inclination, subsequently corrected to a 500 km circular parking orbit. Energy is dissipated by drag during a single atmospheric pass; the bank angle rotates the lift vector to manage how much energy is shed and in which plane.

The natural state space for the maneuver is the (orbital energy, dynamic pressure) plane (Figure 1). The vehicle enters with positive orbital energy (hyperbolic, $E \approx +4.9$ MJ/kg) and must exit with negative energy (bound, $E \approx -5.9$ MJ/kg) without either skipping back out – a hyperbolic escape – or descending so deep that it crashes or violates a thermal limit. The set of trajectories that capture is bounded by two constant-bank profiles: an overshoot boundary (full lift-up, the limit against escape) and an undershoot boundary (full lift-down, the limit against crash). A practical **restricted corridor** tightens these to the bank profiles that land within a bounded apoapsis error of the target. Control authority is proportional to dynamic pressure, so the guidance has the most leverage deep in the pass and almost none on the thin entry and exit legs. The vehicle must respect a peak heat-flux limit of 200 kW/m², a 4 g load limit, and an integrated heat-load limit of 25 MJ/m².

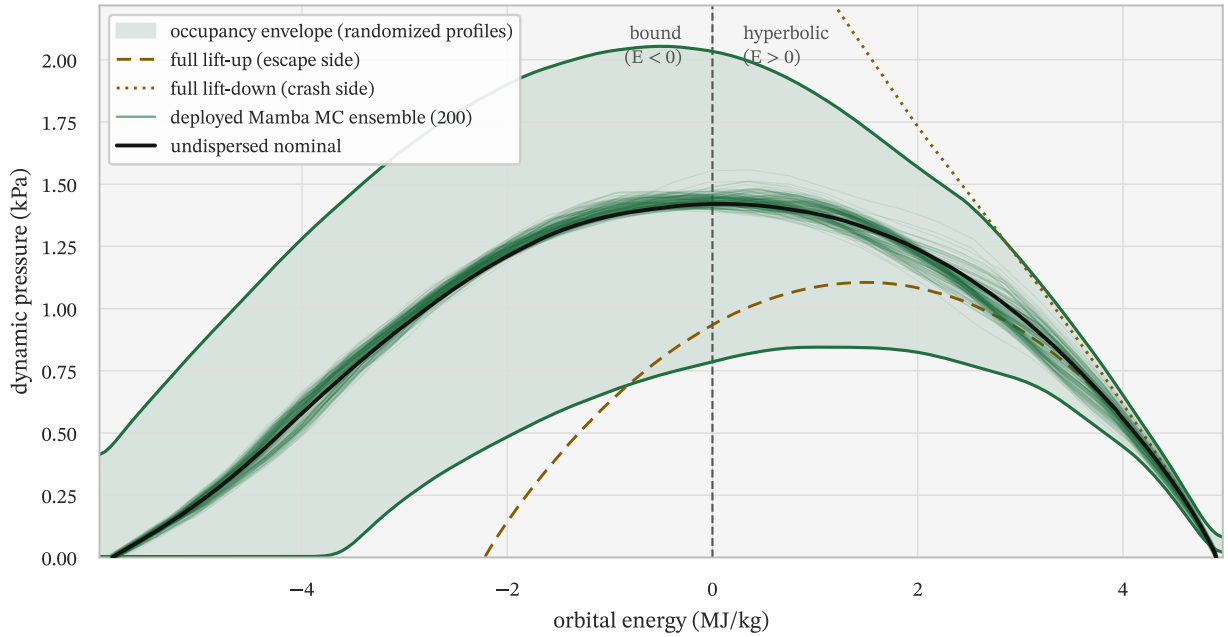


Figure 1: Empirical trajectory-occupancy envelope of the aerocapture corridor in the (orbital energy, dynamic pressure) plane, traced by a $1,000,000$-run dispersed Monte Carlo of randomized signed piecewise-constant bank profiles (roll reversals included), with the undispersed full-lift-up and full-lift-down constant-bank boundary traces overlaid (dashed). The shaded band spans the occupied corridor: the upper edge is the $p_{99.9}$ dynamic pressure of all capturing trajectories (the crash-side limit), the lower edge the $p_{0.5}$ of trajectories capturing below a 5000 km apoapsis (the escape-side limit); as quantiles of sampled profiles these edges are empirical, not a formal reachable set. The vehicle enters hyperbolic ($E > 0$, right) and bleeds energy into a bound orbit ($E < 0$, left); the 200-run Monte Carlo ensemble of the development-regime Mamba champion and its undispersed nominal (heavy line) fly well inside the envelope.

2.2 The objective: the tail that sizes the mission

Performance is measured by the **correction Δv** – the propellant cost to carry the captured orbit to the 500 km circular parking orbit, summed over the periapsis-raising, circularization, and plane-change burns. At atmosphere exit the periapsis always sits inside the atmosphere and must be raised, so the correction cost has a floor of roughly 105 m/s at this entry interface, set by the nominal periapsis raise (dispersed cases spread a few m/s around it; the 2009 study's interface put the same floor near 113 m/s); a guidance law cannot do better than deliver the vehicle to that floor across all dispersions. We define a run as a **capture** when it terminates in a bound orbit (the simulator's terminal flag $i_{\text{final}} = 3$ with eccentricity < 1) and compute Δv over captured runs only.

The point we want to make sharply, because it governs every comparison in this paper, is that the **mean** correction cost is operationally almost irrelevant. Aerocapture propellant tanks are sized for the worst credible case, conventionally the 3σ design point ($\approx$ the 99.87th percentile), not the median. Two guidance laws with the same mean but different tails are not equally good: the one with the heavier tail forces larger tanks and a heavier, more expensive mission. We therefore treat the **tail** of the Δv distribution as the objective, and report the conditional value-at-risk – the mean cost in the worst $(1-\alpha)$ fraction of cases, CVaR_α (Rockafellar and Uryasev, 2000) – at $\alpha = 95\%$ and, for sizing decisions, at the far-tail depth $\mathrm{CVaR}_{99.9}$, together with the 95th and 99th percentiles and the sample maximum (a descriptive bound). Empirically CVaR_α is the mean of the worst $\max(1, \mathrm{round}((1-\alpha)n))$ captured observations, without interpolation; every tail statistic is reported with the number of observations it averages. The far tail cannot be estimated from a 1000-case ensemble (a single sample beyond $p_{99.9}$), so every sizing number in this paper is computed on a pre-registered pool of 10^6 scenarios, training-disjoint by construction, and the shallower $n =$

1000 and $n = 10,000$ pools carry the CVaR_{95} statistics. We lead with the tail; the mean is reported for continuity with the 2009 work.

Table 1: The dispersion model. Twenty-six static draws across nine domains, plus a time-varying Gauss–Markov density perturbation (the tenth row). The medium regime uses medium presets except for the winds and density perturbation (low); the wind model itself is disabled in this campaign, so its two draws are inert. Gaussian dispersions are quoted at 3σ; uniform dispersions at their full range.

Domain	Dims	Distribution	Dispersion (medium regime)
Entry state	6	Gaussian	altitude ± 0.3 km, velocity ± 3 m/s, flight-path/azimuth ± 0.15–$0.3°$ (3σ; medium)
Atmospheric density	1	Uniform	$\pm 50\%$ multiplicative bias (medium)
Aerodynamics	3	Uniform	drag $\pm 5\%$, lift $\pm 10\%$, angle of attack $\pm 1°$ (medium)
Navigation errors	7	Gaussian	altitude ± 2 km, horizontal $\approx \pm 9$ km per axis, velocity ± 1.2 m/s, drag-accel ± 0.3 m/s^2 (3σ; medium)
Mass	1	Uniform	$\pm 1\%$ (medium)
Vehicle	2	Uniform	reference area $\pm 2\%$, max bank rate $\pm 10\%$ (medium)
Pilot dynamics	3	Uniform	time constant / damping / frequency $\pm 10\%$ (medium)
Nav-filter gain	1	Gaussian	± 0.3 absolute (3σ; medium)
Winds	2	Uniform	speed $\times [0.7, 1.3]$, direction $\pm 5°$ (low; model disabled – draws inert)
Density perturbation	process	Gauss–Markov	correlation time 120 s, 5% RMS, time-varying, one realization per scenario (low)

The mission is flown under 26 dispersed parameters across nine domains, summarized in Table 1, plus a time-varying Gauss–Markov (Ornstein–Uhlenbeck) density perturbation that evolves during each run. The medium regime uses medium presets for the initial-state, atmospheric, aerodynamic, navigation, mass, vehicle, pilot, and navigation-filter domains and low presets for the winds and the density perturbation. The atmospheric density bias alone spans $\pm 50\%$ – it is the dominant driver of apoapsis error, and a guidance law blind to it cannot reject it. The simulator's wind model follows a parametric Mars profile (Forget *et al.*, 1999) but is disabled for this campaign (its two draw dimensions are carried but inert); the density perturbation is an Ornstein–Uhlenbeck process layered on the static bias. Within a multi-scenario batch, draws are generated by Latin-hypercube sampling (McKay, Beckman and Conover, 1979) for space-filling coverage; the evaluation pools draw one scenario per seed and are therefore plain independent samples – which is what the replicate- and bootstrap-based intervals of Sections 6–7 assume.

The noise model is the point to fix before any number. A scenario is its 26 static draws **and** its own realization of the Ornstein–Uhlenbeck process: the process's stream is seeded from a hash of the scenario's dispersion draw, so identical draws reproduce and distinct scenarios receive distinct density histories, and every pool of this paper – training, validation, evaluation, the 10^6 confirmatory – marginalizes over the perturbation. One exception is named once and kept out of every table that mixes with it: the **development regime**, in which all scenarios of a pool share a single realization of the perturbation (time-varying within each run, constant across runs). The methodology and optimizer studies of Sections 4–5 and the architecture probes of Appendix B were run in it; they compare like with like under one conditioning, and their numbers are development quantities. Section 6.4 measures what the conditioning does to a trained policy.

3 Guidance schemes

We benchmark the neural policies against six classical schemes that span the spectrum of aerocapture guidance, from analytic feedback through numerical prediction. All schemes share the same simulator, the same navigation

chain, and – where they need one – the same kind of reference trajectory, so the comparison isolates the guidance law itself (Table 2).

3.1 Classical schemes

Feedback Trajectory Control (FTC) is the Cerimele–Gamble-derived scheme from the 2009 study (Cerimele and Gamble, 1985; Gelly and Vernis, 2009) and our continuity baseline. It builds a virtual reference trajectory on apoapsis control alone – a constant-bank profile that reaches the target apoapsis without inclination control, tabulated as $\cos\mu_{\text{ref}}$, $\dot{h}_{\text{ref}}$, and q_{ref} versus orbital energy – and enslaves the commanded bank to it through a proportional law on the altitude-rate and dynamic-pressure errors,

$$\cos\mu_{\text{com}} = \cos\mu_{\text{ref}} + G_{\dot{h}}\frac{\dot{h}-\dot{h}_{\text{ref}}}{q} + G_q\frac{q-q_{\text{ref}}}{q}, \tag{1}$$

where q is the dynamic pressure and $G_{\dot{h}}$, G_q are the altitude-rate and dynamic-pressure feedback gains; the command is clamped to 0° or 180° when $|\cos\mu_{\text{com}}| > 1$. It decouples in-plane (apoapsis) from out-of-plane (inclination) motion, the latter handled by a roll-reversal logic that flips the bank sign whenever the inclination error, projected a few seconds ahead, exceeds a threshold inside an energy-gated activation window. FTC is analytic, fast, and – as we will show – only as good as its reference.

FNPAG is Lu's fully numerical predictor–corrector (Lu, 2014; Lu *et al.*, 2015), subsequently carried across Mars aerocapture mission and vehicle design maps (Matz *et al.*, 2017). Every two-second replan cycle it integrates the equations of motion forward to atmosphere exit and bisects the constant capture-bank magnitude until the predicted osculating exit apoapsis matches the target, scaling the onboard atmosphere by the navigation-estimated density factor so the predictor tracks the measured atmosphere rather than a nominal model; the command is held between replans. It is the most accurate classical scheme and, at roughly eleven forward integrations per replan, by far the most expensive.

PredGuid is the Apollo/Shuttle-heritage drag-tracking law (Harpold and Graves, 1979; Bairstow, 2006): it tracks a drag-versus-energy reference profile with negative feedback.

Energy controller tracks an energy-dissipation reference through dynamic-pressure and altitude-rate feedback.

Equilibrium glide holds the equilibrium-glide condition with altitude-rate damping and a velocity bias, using the navigation-filtered density rather than a static table.

Piecewise constant flies an N-segment constant-bank profile; it is the simplest scheme, produces the reference trajectory and corridor the other schemes consume, and – like the full-neural network – emits a **signed** bank, so it bypasses the shared roll-reversal, exit, and thermal-limiter logic.

3.2 Neural guidance

The neural policy maps an observation vector to a bank command. We generalize the 2009 single-hidden-layer feed-forward network – five hand-picked inputs (orbital energy, eccentricity, inclination, velocity, non-gravitational acceleration) feeding a two-output bank decoder – along three independent axes: the **inputs** the policy reads, the **bank decoder** it writes through, and the **cell type** that carries state across the pass. We take them in turn.

3.2.1 Inputs

The modern policy draws from a 35-element candidate vector; a learned input mask selects the subset that actually reaches the network (the deployed atan2 policies use 17). Sixteen entries are instantaneous orbital, aerodynamic, and thermal state variables. The other nineteen are **engineered** signals that hand the policy temporal context and a cost-aligned summary of the flown trajectory, so it need not integrate the history itself. A few are genuinely autoregressive – seam-free $(\sin,\cos)$ encodings of the recent bank history and the roll-reversal telemetry – but most are instantaneous functions of the current osculating orbit: reference-trajectory interpolations ($\dot{h}$ and q at the current energy), a closed-form exit-bank teacher signal, the periapsis altitude, and – most important, as the ablation of Section 8 shows – the three **predicted correction-Δv** components (the energy-closing, periapsis-correction, and

plane-change burns). Those last three are smooth, causal, and cost-aligned: they tell the network what the maneuver would cost if it stopped now.

3.2.2 Bank decoder

The 2009 paper read the bank from a two-element output,

$$\mu = \operatorname{atan2}(o_1, o_2), \tag{2}$$

which we keep as the default (`atan2_signed`). It wastes half its range when only the magnitude is needed, so for magnitude-only policies the `acos_tanh` decoder maps a single output smoothly onto $[0, \pi]$,

$$\mu = \arccos(\tanh(o_1)). \tag{3}$$

Two further single-output decoders attack the $\pm\pi$ wrap seam a raw angle output suffers near $\mu = \pi$. The `scaled_pi` decoder pushes the seam out of the operating region,

$$\mu = \operatorname{wrap}_{\pi(n\pi\tanh(o_1))}, \tag{4}$$

while the `delta` decoder applies a bounded increment on the previous realized bank,

$$\mu = \operatorname{wrap}_{\pi(\mu_{\text{prev}}+\Delta_{\max}\tanh(o_1))}. \tag{5}$$

3.2.3 Cell type

Where 2009 had a single hidden layer, we span six architecture families behind one common runtime, each carrying a different kind of internal state across the atmospheric pass:

- **Dense** feed-forward – memoryless; maps the current observation straight to a command. The 2009-style baseline.
- **GRU** (Cho *et al.*, 2014) – a gated recurrent cell carrying one hidden-state vector, updated each tick through reset and update gates.
- **LSTM** (Hochreiter and Schmidhuber, 1997) – a recurrent cell that adds a separate long-term cell state alongside the hidden state, regulated by input, forget, and output gates.
- **Windowed** – a zero-parameter FIFO buffer of the last N observations flattened into a dense stack: explicit recent history rather than a learned state.
- **Transformer** (Vaswani *et al.*, 2017) – a causal-attention block attending over a fixed-length key/value window of recent steps.
- **Mamba** (Gu and Dao, 2023) – a selective state-space core: a linear recurrence whose gates depend on the input, carrying a compact SSM state.

We use **recurrent** loosely for any cell that carries internal state across the pass. All six train and deploy through the same bit-validated Rust runtime, and all are sized so the comparison across cell types holds the parameter budget roughly fixed. Appendix B probes three further recent recurrent families – closed-form continuous-time cells (Hasani *et al.*, 2022), the exponential-gated and matrix-memory xLSTM variants (Beck *et al.*, 2024), and the discretization and complex-state axes of Mamba-3 (Lahoti *et al.*, 2026) – against these cells at matched budget; none improves on them.

Table 2: The benchmarked schemes. "Reference" marks dependence on a tabulated reference trajectory ("inputs": the network reads two reference interpolations as observations but does not enslave to them); "Compute" classes are quantified in Section 7.2 (fast: 1–3 ms/sim; slow: 208 ms/sim). Signed-bank schemes (full-neural, piecewise-constant) bypass the shared roll-reversal, exit-phase, and thermal-limiter logic.

Scheme	Bank command	Reference	Compute	Heritage / note
FTC	magnitude + roll reversal	yes	fast	Cerimele–Gamble apoapsis enslavement
FNPAG	magnitude + roll reversal	no	slow	onboard forward integration, bisection corrector
PredGuid	magnitude + roll reversal	yes	fast	Apollo/Shuttle drag tracking
Energy controller	magnitude + roll reversal	yes	fast	energy-dissipation tracking
Equilibrium glide	magnitude + roll reversal	no	fast	equilibrium-glide condition, nav density
Piecewise constant	signed (N segments)	no	fast	produces reference + corridor
Neural network	signed or magnitude	inputs	fast	35-input candidate vector, stateful cells

4 Training methodology

The training methodology is the load-bearing contribution. The policies are trained without input–output pairs or any reference-tracking objective (two observation inputs interpolate the reference table; nothing enslaves to it): each candidate network is simulated on a batch of dispersed Monte Carlo scenarios, and its fitness is the resulting correction-Δv cost (with soft constraint penalties), so the optimizer searches directly on mission performance. The same recipe trained the 2009 networks. What changed – and what makes the modern policies work – is the realization that **how the scenario batch is chosen, generation to generation, matters more than the optimizer or the architecture**. The right environment is non-stationary: it keeps moving the scenarios beneath the population so that no individual can memorize a fixed set. Four design choices – a genetic algorithm, an adaptive (moving) seed schedule, a tail-weighting cost transform, and a hardest-case seed curation – form one matched system. We take them in turn. The seed-strategy, cost-transform, curation and allocation studies of this section and the optimizer grids of the next were run in the development regime of Section 2.2 (one density-noise realization shared across the scenarios of a pool); they compare like with like under that conditioning and their numbers are development quantities. The reinforcement-learning baseline of Section 5 and everything from Section 6.2 on are per-scenario, save the bank-decoder comparison that closes Section 6.4.

4.1 Seed strategy: the genetic algorithm needs a non-stationary objective

The single largest effect in the entire campaign is the seed strategy. We compared three: **fixed** (a deterministic scenario batch, identical every generation), **rotating** (a fresh random batch every generation), and **adaptive** (a curated batch, refreshed on a validated improvement or periodically, drawn from the cost distribution of the best individuals). The decisive cells carry three independent training seeds each. Under fixed seeds the genetic algorithm is the **worst** optimizer we tested and an unstable one – three trainings land at 160.3, 166.9, and 133.5 m/s mean (CVaR_{95} 215.8 on the first seed) – because it overfits the repeated scenarios, evolving a policy that is excellent on those particular draws and mediocre elsewhere. Rotating the seeds rescues it outright and **reliably**: 119.4 m/s three-seed mean with a 3.5 m/s seed range (first seed 120.0 mean, 144.5 CVaR_{95}) – a 34 m/s drop in three-seed means, a 71 m/s drop on the tail on the first seeds, and the two distributions do not touch: the fixed schedule's luckiest seed sits 12.5 m/s above the moving schedule's worst. Adaptive curation is indistinguishable from rotating on the **mean** at this allocation (119.6 versus 119.4, seed ranges overlapping; the first-seed 118.0 was a favorable draw); its contribution is the hardest-seed tail shaping of Section 4.2, which a plain rotating schedule cannot express.

The control that makes this a statement about the **objective** and not about compute is CMA-ES. Run on the identical fixed-versus-rotating change, CMA-ES does not move – 125.7 versus 126.1 m/s in three-seed means, seed ranges within ± 1 (first seeds 126.9 $\rightarrow$ 127.3) (Figure 2). Under fixed seeds the genetic algorithm's selection converges onto the quirks of those particular draws; CMA-ES neither suffers under the fixed batch nor benefits from the moving one. We state that asymmetry empirically rather than mechanistically: a plausible reading is that CMA-ES's continuous re-estimation of its parameter-space search distribution already decorrelates successive generations, while scenario noise chiefly perturbs its rank-based updates and step-size control – consistent with its self-termination on the noisy objective (Section 5) – but our experiments were not designed to isolate the mechanism. Rotating costs marginally more compute, but CMA-ES given the same marginal compute gains nothing, so the lever is the non-stationarity of the objective, not the extra evaluations. The practical consequence is striking: under a fixed objective one would deploy CMA-ES and discard the genetic algorithm; under a moving objective the genetic algorithm becomes the best optimizer in the study. The moving environment does not make the genetic algorithm **robust** to a moving objective – it makes the genetic algorithm **need** one.

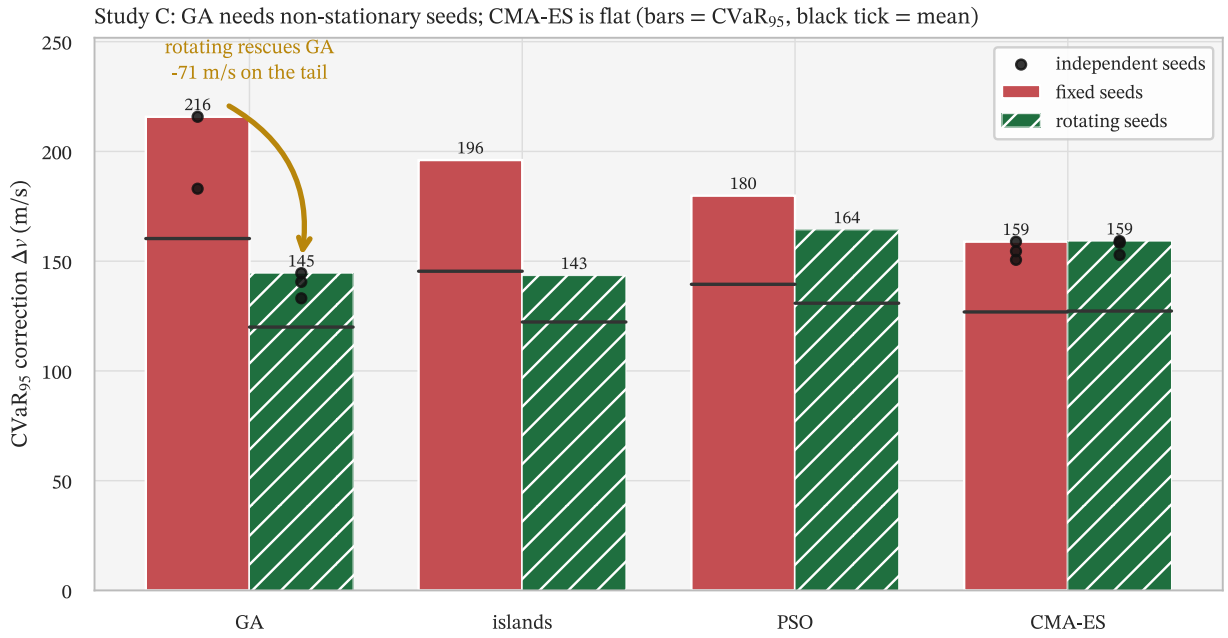


Figure 2: Fixed versus rotating seeds, per optimizer. The genetic algorithm is the worst optimizer under fixed seeds and is rescued by rotating them (-34 m/s in three-seed means, -71 m/s at CVaR_{95} on the first seeds); CMA-ES is essentially unchanged (candidate mechanisms are discussed in the text). Black dots: three independent training seeds for the repeated GA and CMA-ES cells – the fixed-seed pathology is itself high-variance. The lever is the non-stationary objective, not the extra compute.

4.2 Cost transform, curation, and allocation

The same worst-case-leaning logic that makes us report the tail also shapes how we train. Because the mission is sized off the far tail, we apply a monotonic **cost transform** to each per-simulation cost before aggregating, so that the optimizer feels expensive scenarios more sharply. Applied per scenario before the root-mean-square aggregation over the individual's batch (Appendix A), the cubed transform makes the per-individual objective

$$J = \left(\frac{1}{n} \sum_{i=1}^{n} C_i^6 \right)^{1/2}, \tag{6}$$

a monotone function of the L_6 norm of the per-scenario cost vector – a high-moment objective that weights an individual's worst scenarios far more heavily than its typical ones. (For a single scenario a monotone transform is ranking-neutral; across a batch it deliberately is not.) We prefer this smooth high-moment proxy over optimizing an empirical tail quantile directly because the deployed allocation evaluates as few as two scenarios per individual per generation – far too few to estimate a quantile – while the moving batch supplies tail coverage across generations. Evaluated at the depth that matters – a far-tail $n = 10,000$ pool – the cubed transform compresses the extreme tail best ($\mathrm{CVaR}_{99.9}$ 153.0 m/s, sample max 160.1) against linear (156.7/162.2), square-root (158.4/167.1), squared (162.7/180.9), and logarithmic (162.3/180.6). The logarithm is worst across the shallow and mid tail because it over-compresses and starves the gradient between captures; only at the very extreme is the squared transform worse still. These are single training runs, and the cubed-versus-linear margin (3–5 m/s) is within the run-to-run scatter we measure in Section 6, so we read the direction – deeper tail-weighting pays at deeper sizing depth – rather than the exact ranking as the finding. A shallower metric (CVaR_{95}) would have mildly favored square-root; the deeper we look into the tail, the more the tail-weighting pays, which is exactly why the sizing depth must decide (Figure 3).

Seed **curation** is the same mechanism applied to the scenarios rather than the cost. At each refresh the adaptive strategy bins the cost distribution of the best individuals into quantiles and picks one representative seed per bin; the choice of representative is the lever. Picking the **hardest** seed per bin (the "max" bucket) dominates the far tail ($\mathrm{CVaR}_{99.9}$ 153.0 m/s) against the bin median (193.9), a random pick (173.1), and the **easiest** seed (225.9). The easiest-seed bucket is the cautionary tale: the best mean, 117.8 m/s, and a catastrophic worst case – it drops captures and reaches 245 m/s, optimizing the average at the expense of the worst case. Trimming the cost distribution helped nothing (Figure 5). These too are single runs, but the min- and middle-bucket gaps (40–70 m/s) sit far outside run scatter. The cost transform and the curation bucket are therefore one idea – force the policy onto the hard cases – expressed twice.

Finally, **allocation**: given a compute budget, is it better spent on more scenarios per generation or more generations? Under rotating seeds with a fixed generation count the sweet spot is moderate ($n_{\mathrm{sims}} = 10$). But under the adaptive strategy with the budget spent on generations, the answer flips hard: just $n_{\mathrm{sims}} = 2$ per generation, run for many generations, gives the best mean (109.9 m/s) and CVaR_{95} (117.5), beating 5, 20, and 100 (Figure 4). The few-sample noise per generation is bought back, and then some, by the far greater scenario diversity the moving environment sees over the run. The deployed headline policy uses this allocation.

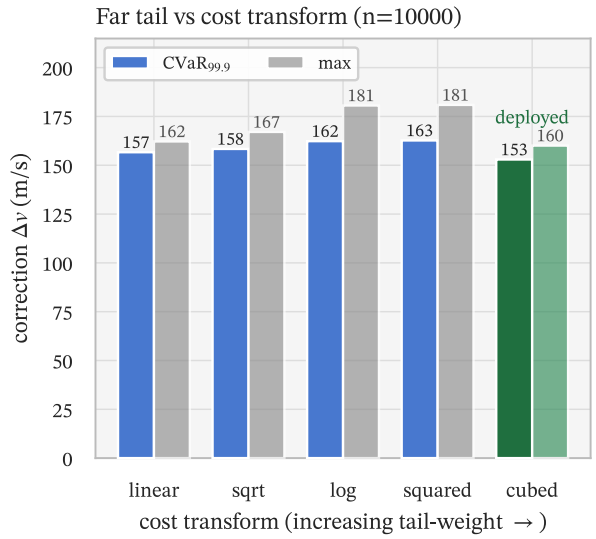


Figure 3: Cost transform versus the sizing tail, at the sizing depth ($n = 10,000$ pool). The cubed transform minimizes both the far-tail $\mathrm{CVaR}_{99.9}$ and the sample maximum, even though a shallow CVaR_{95} read would mildly favor square-root.

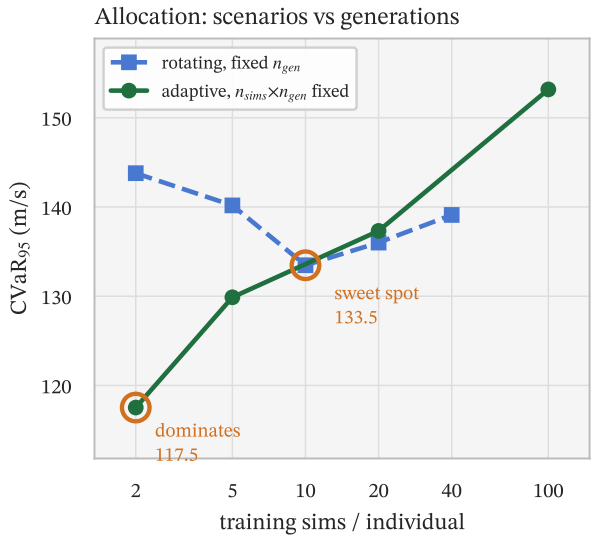


Figure 4: Allocation of the compute budget (CVaR_{95}; the mean orders both series identically). Under the adaptive schedule, two scenarios per generation over many generations dominates larger per-generation batches.

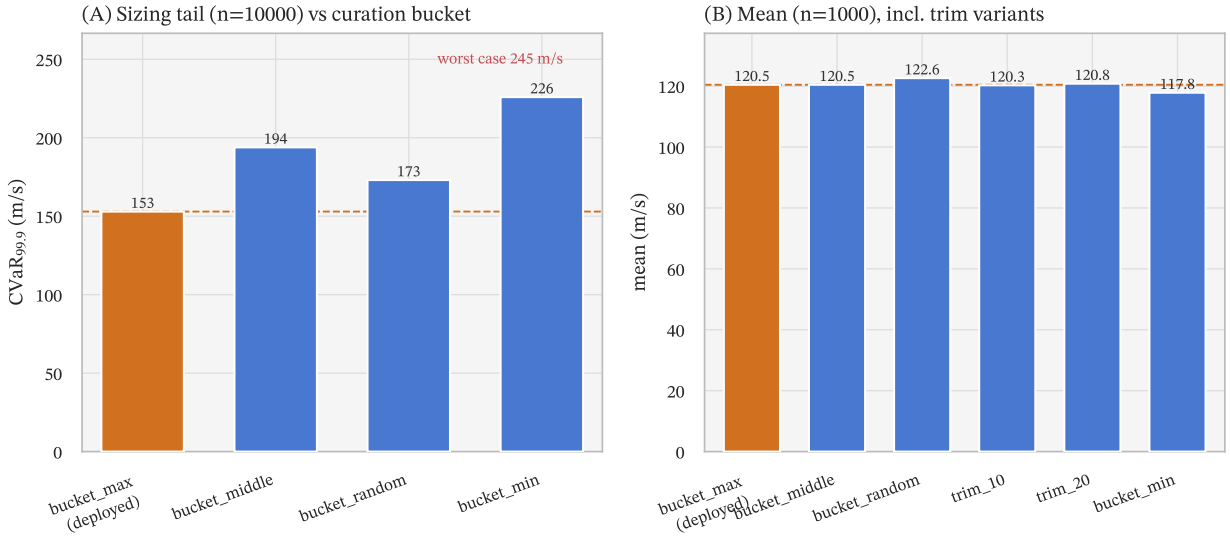


Figure 5: Seed curation. Left: far-tail $\mathrm{CVaR}_{99.9}$ ($n = 10,000$) per curation bucket – selecting the hardest seed per cost-CDF bin (the "max" bucket) compresses the sizing tail, and the easiest-seed bucket blows it up (worst case 245 m/s). Right: mean ($n = 1000$) – the easiest bucket wins the mean, the optimize-the-average trap; the trim variants match the max bucket and were not carried to the far-tail pool.

That the optimal allocation pours the budget into generations points to the last methodological fact: training here is **compute-bound, not overfitting-bound**. A stationary objective eventually overfits, and one stops early. The moving objective never converges to a fixed landscape, so the validation error – the RMS on the reserved selection pool of Table 3; we keep the conventional name for the loss curve – keeps falling for nearly the whole twenty-thousand-generation run and then plateaus rather than degrading, i.e. the policy does not memorize scenarios

(Figure 6). The plateau also exposes a counter-intuitive dimensionality effect that recurs in the architecture results: the 972-parameter dense network learns **faster** early – more plasticity – but the 515-parameter network overtakes it and plateaus **lower** (validation RMS 1.326×10^6 versus 1.433×10^6). For the gradient-free genetic algorithm, the extra dense parameters are more search burden than added capacity; the "more parameters, learn faster" intuition does not transfer.

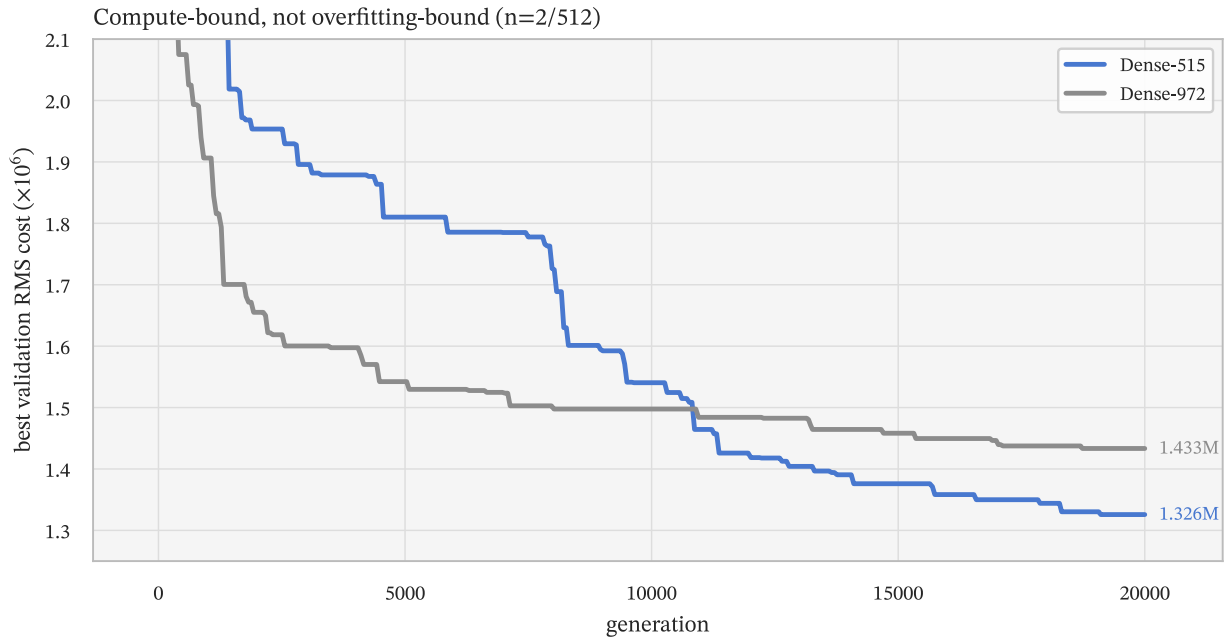


Figure 6: Best validation RMS versus generation for the two dense reference networks. Both keep improving until late in the twenty-thousand-generation run and then plateau – under the non-stationary objective the policy generalizes across the dispersion distribution rather than memorizing scenarios. The 515-parameter network plateaus below the 972-parameter one: beyond a few hundred parameters, extra dense capacity hurts the gradient-free search.

Table 3: Scenario-pool roles, the noise regime each was run in, and the decisions each pool influenced. The selection pool is adaptively reused by the promotion gate and the development far-tail pool informed the methodology choices, so neither is an unbiased test set. The confirmatory pool was generated from a seed range disjoint from every earlier draw, after all methodology, architecture, and checkpoint choices were frozen, and each cell was evaluated on it exactly once; the deployed seed of every cell was fixed by rule before the pool was queried.

Pool	n	Noise	Queries	Decisions taken on it
Training batches	2/gen	per-scenario	every generation	weight updates (moving, curated)
Selection (offset 1M)	1000	per-scenario	$\approx 13,000$ per run	in-training argmin promotion
Development far tail (offset 2M)	$10,000$	shared path	tens	cost transform, curation bucket, allocation (Section 4)
Final evaluation (offset 2M)	1000	per-scenario	once per cell	none – the paired CVaR_{95} tables
Fresh re-quote (offset 8M)	1000	per-scenario	once; the grid	quantization cell choice (Appendix C)
Confirmatory sizing (App. A)	$10 \times 100,000$	per-scenario	once, post-freeze	none – every quoted sizing number
Off-nominal stress (offset 9M)	$10,000$	per-scenario	once per policy	none (robustness probe, Section 7)
Architecture probes (offset 10M)	1000	shared path	once per arm-repeat	none (Appendix B verdicts)

The studies of this section and the next are exploratory – single training runs in the development regime unless stated otherwise; the confirmatory statements of this paper are the per-scenario frozen-pool quantities of Sections 6–7.

5 Optimizer and dimensionality

Every optimizer in this study is a gradient-free population method, a deliberate choice rather than an omission. The training objective is the correction Δv of a complete atmospheric pass – produced by a simulator with discrete capture, crash, and atmosphere-exit termination, threshold-triggered phase transitions, and hard constraint limits – so it is a black-box function of the network weights with no usable gradient. Policy-gradient reinforcement learning (PPO (Schulman *et al.*, 2017), SAC (Haarnoja *et al.*, 2018)) does not require a differentiable simulator or a differentiable reward – it estimates gradients from sampled rollouts and can in principle optimize a terminal-only objective. We trained PPO under the protocol of the population cells it is compared against: the same 17-input observation contract, per-input normalization and `atan2` decoder, the same architecture and co-tuned navigation and command-shaping scaffolding as the 515-weight dense and 1014-weight GRU cells of Section 6 (the GRU trained by PPO with truncated backpropagation through time), per-scenario noise, a reserved training pool (offset 3×10^6), promotion on the reserved selection pool and the quote on the reserved final-evaluation pool, paired against the population cell (Table 4). The reward is potential-based per-step shaping aligned to the predicted correction cost plus the true terminal cost, the budget is 30×10^6 environment steps per cell, the deployed artifact is the best validation checkpoint, and the environment flies exactly what the deployed network flies (a bit-identity gate holds the two loops together). From scratch, PPO lands 73–330 m/s above the dense population cell in paired mean over three torch seeds and 108–338 above the GRU one, the seed-to-seed spread as large as the gap, and every scratch cell still improving at the budget. Warm-started from the population champion, PPO deploys the champion (the first validation checkpoint; paired mean -0.2 and $+0.4$ m/s) and then walks off it, consistent with the stochastic shaped return optimizing a different quantity than the deterministic mission cost. Population search on the mission cost itself was simply the stronger tool here, so throughout we optimize the mission cost directly.

Table 4: PPO against the population-trained cell of the same architecture (the 515-weight dense and 1014-weight GRU cells), correction Δv in m/s. Capture, mean and CVaR_{95} on the $n = 1000$ final-evaluation pool; Viol. is the share of the 10^6 confirmatory pool exceeding any constraint (the population trainer's feasibility gate has no counterpart in the RL loop: the unseeded dense scratch run bought its capture with heat flux); the deltas are paired against the population cell (negative favors PPO) with a 10,000-resample bootstrap 95% interval on the mean. Seed 1 of each scratch cell ran before the torch seed was pinned; seeds 2 and 3 are repeats.

PPO cell	Capture %	Viol. %	Mean	CVaR_{95}	Δmean vs pop.	$\Delta\mathrm{CVaR}_{95}$	PPO-win %
Dense, scratch, s1	100.0	26.56	443.6	575.6	+330.2 [+326.2, +334.1]	+451.5	0.0
Dense, scratch, s2	100.0	0.12	186.4	223.9	+73.0 [+71.8, +74.2]	+99.7	0.0
Dense, scratch, s3	100.0	0.70	195.9	262.2	+82.5 [+80.4, +84.6]	+138.1	0.3
Dense, warm start	100.0	0.23	113.2	123.7	−0.2 [−0.4, +0.0]	−0.5	54.3
GRU, scratch, s1	100.0	0.01	451.1	568.6	+338.2 [+333.8, +342.4]	+446.5	0.0
GRU, scratch, s2	100.0	2.71	405.6	493.4	+292.6 [+288.7, +296.4]	+371.3	0.0
GRU, scratch, s3	100.0	0.43	220.5	348.0	+107.5 [+104.6, +110.5]	+225.8	0.0
GRU, warm start	100.0	0.01	113.4	122.9	+0.4 [+0.3, +0.5]	+0.7	36.6

Having established that the genetic algorithm is the right optimizer under a moving objective, two questions remain: does it need a population that scales with the search dimension, and does the optimizer choice even matter for the low-dimensional classical-gain problems? We compared the genetic algorithm (Goldberg, 1989) against CMA-ES (Hansen and Ostermeier, 2001), particle-swarm optimization (Kennedy and Eberhart, 1995) and its quantum-behaved variant (Sun, Feng and Xu, 2004), differential evolution (Storn and Price, 1997), and a three-island heterogeneous model, on an optimizer-by-budget grid at the largest dense network (3998 weights) and an optimizer-by-dimension grid spanning the 26-parameter FTC-gain problem and the 515-parameter dense network.

At 3998 weights the population size is decisive (Figure 7). The genetic algorithm is best at a population of 150 (118.0 m/s mean; three-seed 119.6, range 3.0) and 300 (120.5 m/s) – a gap smaller than the run-to-run scatter we measure on retrained cells in Section 6, so we report them as indistinguishable – but at a population of 60 it **collapses** to 166.3 m/s. Sixty individuals cannot cover a four-thousand-dimensional weight space; selection drifts. So the tempting generalization that the genetic algorithm dominates at any budget is wrong: at a starved population it is no better than a single restart. The corrective is simple and worth stating plainly – the population must scale with the search dimension. CMA-ES improves smoothly with budget (133.3 → 126.3 → 121.8 m/s) but never reaches the genetic algorithm's optimum and self-terminates on the noisy objective before exhausting a generous generation count, an asymmetry to keep in mind for compute-matched comparisons. The three-island heterogeneous trainer (particle swarm, genetic, and differential evolution, with periodic migration) is the most budget-robust of all, 120–124 m/s across every budget, but the well-populated single genetic algorithm edges it.

The dimensionality grid carries the more useful lesson. On the 26-parameter FTC-gain problem every gradient-free optimizer we ran lands in a tight band, roughly 170–178 m/s – islands 169.8, particle swarm 170.9, CMA-ES 171.7, quantum-behaved swarm 172.9, differential evolution 178.2 – an 8 m/s spread we do not attempt to rank (Section 9). Optimizer choice barely matters when there are only twenty-six gains to tune. It is at neural-network dimensionality that the optimizers separate: at 515 weights the genetic algorithm is best (117.4 m/s), the islands next (118.6), and particle swarm worst (129.8), a 12 m/s spread. The confound is that the 26-parameter cell is a different guidance scheme (FTC), not a narrowed version of the same network, so the comparison conflates dimension with law; but the direction is clear – the optimizer earns its keep on the high-dimensional weight search, not on low-dimensional gain tuning.

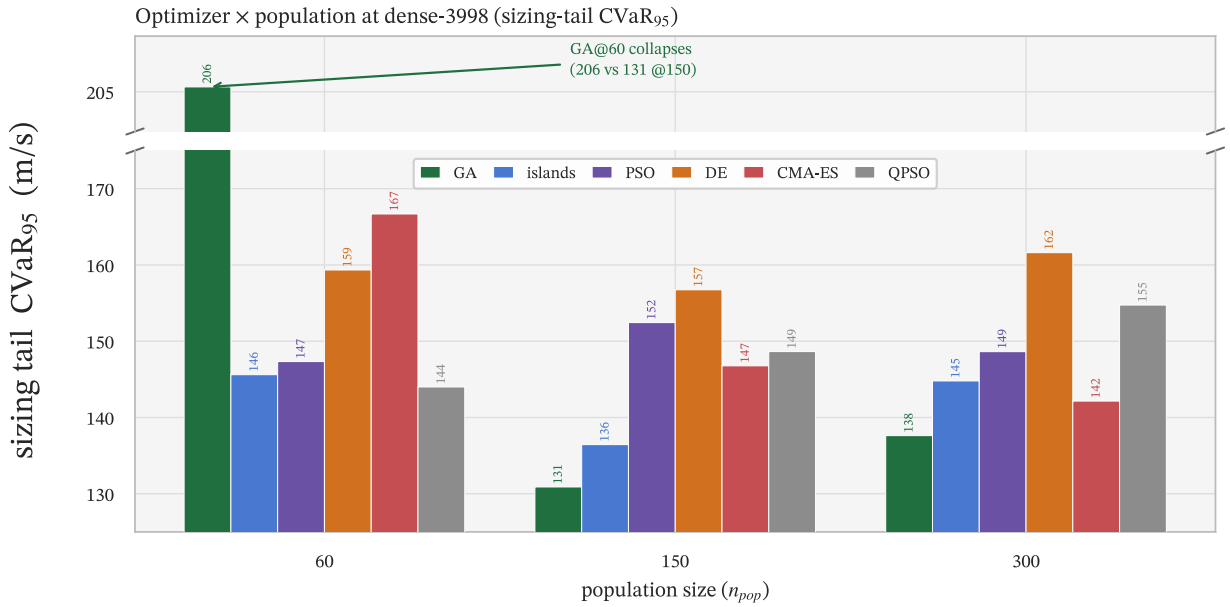


Figure 7: Optimizer by population budget at 3998 weights. The genetic algorithm is best at populations of 150–300 but collapses at 60 (the population must scale with the search dimension); the heterogeneous island model is the most budget-robust; CMA-ES improves with budget but trails the genetic optimum.

The island model – the most budget-robust optimizer above – was built for exactly that robustness (Figure 8). It splits one population budget across three islands running complementary operators (particle swarm, genetic, and differential evolution) on the same scenarios, and every k generations migrates each island's best n individuals into the others, overwriting their worst. Two properties follow. The heterogeneous operators explore the weight space differently and the migration cross-pollinates their discoveries, so a sub-population trapped in a local optimum is pulled out by a migrant from another island instead of having to escape on its own – the diversity a single homogeneous method lacks. And because the three strategies share one run's evaluations through migration, one need not run each optimizer separately and keep the best. The result never collapses the way a starved genetic algorithm does (mean 120–124 m/s across every budget); the price is that at a well-chosen budget a single large genetic algorithm still edges it, so the islands buy robustness to the budget choice rather than a lower optimum.

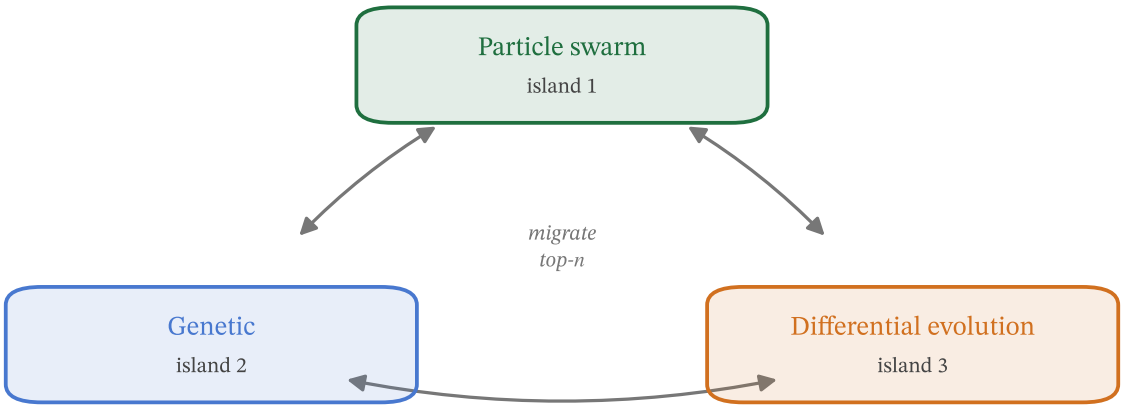


Figure 8: The three-island heterogeneous optimizer. Three islands run complementary gradient-free operators (particle swarm, genetic, differential evolution) on the same scenarios; periodic migration exchanges each island's best individuals for the others' worst. The heterogeneity plus migration is what escapes local optima, and one population budget covers all three searches.

6 Architecture: the headline result

We now sweep the cell type. The finding comes in two halves: a flat median across cell types, and a separation that appears only on the **tail** – between the networks and the classical laws at every depth, and between the networks themselves only inside their run-to-run spread.

6.1 Parameter budget and the capability floor

A parameter-budget sweep across six families (dense, GRU, LSTM, windowed, Transformer, Mamba), each trained identically at two scenarios per generation for five thousand generations in the development regime, makes the first half of the thesis concrete (Figure 9). Every cell in the sweep captures 100% of the time – there is no capability collapse, and a dense network with as few as 102 parameters still guides the maneuver at 100% capture and 120.8 m/s mean. Within the dense family the cost is flat from a few hundred to a few thousand parameters (515 $\rightarrow$ 972 $\rightarrow$ 1957 weights give 117.4 $\rightarrow$ 116.9 $\rightarrow$ 116.8 m/s) and a 3998-weight network gains nothing further – it is over-parameterized for the genetic search, consistent with the plateau result of the training methodology (Section 4). At a matched budget the recurrent and state-space families edge the best dense cell (LSTM 112.0, GRU 112.8, Mamba 114.9 versus the best dense 116.8), while the Transformer pays an attention-overhead penalty at small budgets – its worst cell, at 762 parameters, is the worst in the whole sweep (121.9 m/s), and not for lack of parameters (762 $>$ 515): a small budget forces the attention block to a tiny model width and two dimensions per head, which the genetic algorithm trains poorly. The Pareto front is therefore shallow on the mean; the interesting structure is in the tail, which a five-thousand-generation sweep cannot resolve.

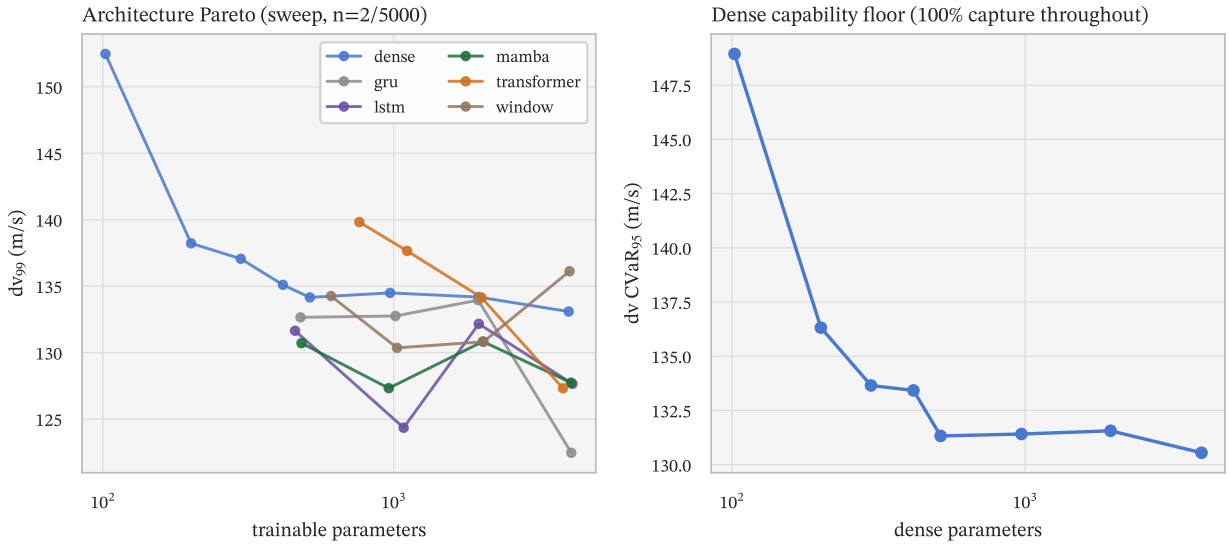


Figure 9: Parameter budget versus sizing tail per architecture family (left) and the dense-family capability floor (right), in the development regime. Every cell captures 100%; the dense cost is flat above a few hundred parameters; recurrent and state-space families edge dense at matched budget; the Transformer is penalized at small budgets. The left panel plots the 99th-percentile cost and the right panel $CVaR_{95}$; the body quotes means, under which the flat cells are indistinguishable.

6.2 The tail at sizing depth

To resolve the tail we trained the strongest recurrent and state-space candidates plus the dense references from scratch at one allocation – two scenarios per generation, a population of 512, 20,000 generations, per-scenario noise, the full methodology of Section 4 – and evaluated each once on the frozen confirmatory pool ($10 \times 100,000$ scenarios). The Mamba and the 515-parameter dense network carry three independent training seeds each; the GRU, the LSTM and the 972-parameter dense network one. Seed 1 of every cell is the deployed one, fixed by rule before the pool was queried; seeds 2 and 3 report the mean and the spread (Table 5, Figure 10).

Table 5: The headline-allocation cells on the confirmatory pool ($10 \times 100,000$ scenarios per cell, per-scenario noise): correction Δv in m/s over captured scenarios, $\mathrm{CVaR}_{99.9}$ with its standard error over the ten replicates, the sample maximum as a descriptive bound. The heat-load column counts the scenarios on which the integrated heat load exceeds the 25 MJ/m² limit; no cell exceeds the heat-flux or g-load limits. The two retuned classical references of Section 7 close the table.

Cell	**Capture %**	**Heat load > limit (per 10^6)**	$\mathbf{CVaR_{95}}$	$\mathbf{CVaR_{99.9} \pm s.e.}$	**Max**
Mamba 962, seed 1 (deployed)	100.0000	1148	120.1	173.9 ± 2.2	300
Mamba 962, seed 2	99.9996	437	131.7	159.3 ± 0.6	621
Mamba 962, seed 3	100.0000	567	122.9	142.3 ± 0.5	266
Mamba 962, three-seed mean ± sd			124.9 ± 6.0	158.5 ± 15.8	
Dense 515, seed 1 (efficiency reference)	100.0000	1465	123.4	140.1 ± 0.4	243
Dense 515, seed 2	99.9999	2112	127.6	185.8 ± 0.6	255
Dense 515, seed 3	99.9929	2572	129.0	155.6 ± 1.2	827
Dense 515, three-seed mean ± sd			126.7 ± 2.9	160.5 ± 23.2	
GRU 1014	100.0000	159	122.8	147.8 ± 0.9	287
Dense 972	99.9999	2670	127.1	170.0 ± 0.6	330
LSTM 1082	100.0000	46	132.7	174.6 ± 1.0	349
FTC, joint reference (retuned)	100.0000	0	147.6	177.8 ± 0.5	242
FNPAG (retuned)	99.9762	10	144.9	189.1 ± 1.8	691

Read family by family, the networks beat the classical laws at sizing depth. The Mamba's three seeds average $\mathrm{CVaR}_{99.9} = 158.5 \pm 15.8$ m/s, the dense reference's 160.5 ± 23.2, and the single-seed GRU, dense-972 and LSTM cells land in the same range (140–186 across the nine runs), against 177.8 for the best classical scheme and 189.1 for FNPAG: eight of the nine network seeds sit below joint-FTC and all nine below FNPAG. Read seed by seed, the ordering between architectures does not survive. The two three-seed means sit within either family's seed standard deviation of each other, the single-seed cells sit inside both spreads, and the deployed Mamba seed – the lowest validation loss of its three and the best CVaR_{95} of the nine (120.1) – has the worst far tail of its family (173.9, 3.9 m/s under joint-FTC). The development-regime ordering we reported earlier (Mamba below LSTM below dense, with seed ranges that barely touched) was a property of the shared density history (Section 6.4); under per-scenario noise the seed spread is the architecture effect's size, and a sizing engineer should quote the family mean with its spread, not one seed. The sample maximum, as before, retires as a comparison statistic at this depth: at 10^6 scenarios it is one draw from the extreme tail (seed 2 of the Mamba, the second-best $\mathrm{CVaR}_{99.9}$ of its family, logged a 621 m/s excursion).

Feasibility is reported beside every tail number because it is no longer free. Every network exceeds the integrated heat-load limit on some scenarios of the 10^6 pool – 46–2670 per million across the nine cells, 1148 for the deployed seed – where the retuned joint-FTC exceeds nothing and FNPAG exceeds it on 10; no cell exceeds the heat-flux or g-load limits. The training gate holds every candidate to zero violations on its $n = 1000$ selection pool, so these are exceedances the gate cannot see, at the 10^{-5}–10^{-3} level, and the appendix cards (Appendix D) put the deployed seed's worst heat load a fraction of a percent over the limit. We therefore report the share rather than exclude the cell, and we read the ceiling itself as a design margin: fine-tuning the deployed seed and the dense reference under

ceilings relaxed by 10% and 20% moves their CVaR_{95} by $+0.7$ and -3.0 m/s, inside each cell's seed spread (± 5.7 and ± 3.9), so the heat-load ceiling is not what sets the tail, and a policy carries its training ceiling (re-flown under the nominal limit the relaxed policies exceed it on 1.7–16.7% of scenarios). Capture is the other half of the feasibility check: two of the Mamba seeds capture 10^6 of 10^6 scenarios, the third loses 4; the dense seeds lose 1 and 71; FNPAG loses 238 (all physical crashes on individual re-runs without the evaluation timeout).

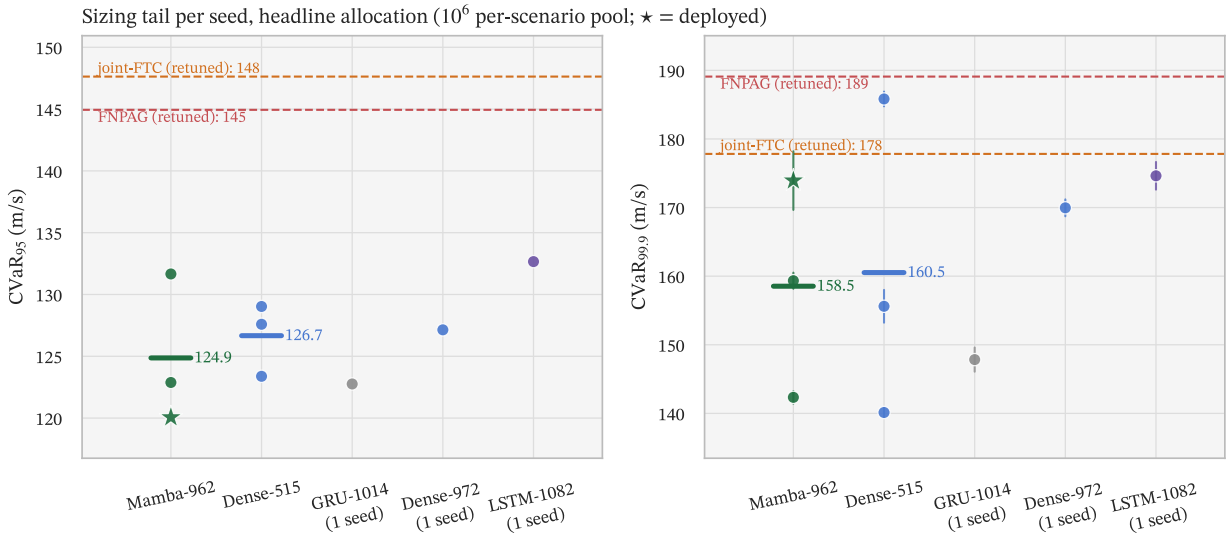


Figure 10: Per-seed CVaR_{95} (left) and $\mathrm{CVaR}_{99.9}$ (right) on the confirmatory pool ($10 \times 100,000$ scenarios per seed, per-scenario noise) for the headline-allocation cells; bars are seed means, whiskers on the right are ± 1.96 replicate standard errors (hidden under the marker where shorter), the retuned joint-FTC and FNPAG are dashed. The deployed cell is seed 1 of the Mamba (starred): the best point on the left panel and the worst of its three on the right. Every family mean sits below both classical references; the families overlap one another.

6.3 What the controls say about state

The mechanism we claimed from the development regime – that genuine internal state buys the tail – is the claim the per-scenario campaign tests most directly, with three controls evaluated once each on the confirmatory pool. **State reset**: the deployed Mamba evaluated with its recurrent state zeroed at every guidance tick captures 99.99% of scenarios but at a $\mathrm{CVaR}_{99.9}$ of 860 m/s against 174, and the whole distribution moves, not only the far tail: CVaR_{95} goes from 120 to 808. The trained policy computes with its state; it is not a feedforward law in disguise. **Matched history**: a dense cell fed an explicit five-tick observation window (970 parameters, no learned state, identical allocation and regime) retrained over three seeds reaches $\mathrm{CVaR}_{99.9} = 178.9 \pm 41.5$ m/s. **No predicted-Δv**: the same Mamba architecture retrained without the three cost-aligned inputs (914 parameters, three seeds) reaches 161.1 ± 18.3. Against the champion family's 158.5 ± 15.8, the window control sits $+20.4$ m/s and the stripped network $+2.6$ (Welch $t = 0.80$ and 0.19 on three seeds a side); the window's mean rests on one seed (226.7), its other two sit inside the champion's range, and its seed spread is the widest of the three families. Neither difference is resolved. The honest statement is therefore narrower than the one we set out to make: the deployed policy depends on its own state (the reset control), but at this allocation and budget neither learned state nor the engineered cost-to-go inputs buys a far-tail advantage that three seeds can see. What the architecture buys is not settled by this campaign; what it costs is (Section 7.2).

Training loss does not pick the tail winner, and the per-scenario runs make the point more sharply than the development regime did. Across the nine headline-allocation runs the best validation RMS – the training objective on the reserved selection pool – is not positively correlated with the far tail (Figure 11): the deployed Mamba seed has the lowest loss of its family and its worst $\mathrm{CVaR}_{99.9}$, and the dense seed with the lowest loss of its family has the widest tail of the nine. The selection gate promotes on that loss, so a deployed seed is the seed the gate liked, and the

far tail is a quantity the gate cannot see; this is why every network cell carries its seed spread and why the deployed seed was fixed by rule rather than selected. The reason the median stays flat is visible from the inside (Section 8): the engineered, cost-aligned inputs already encode most of what a memory cell could recover, so every architecture trained to convergence reaches the same 110–116 m/s typical cost. What remains open is whether more sophisticated memory would help: Appendix B probes three further recurrent families (CfC, the xLSTM cells, Mamba-3's axes) at matched budget in the development regime, and none beats the plain cells – two arms are significantly worse.

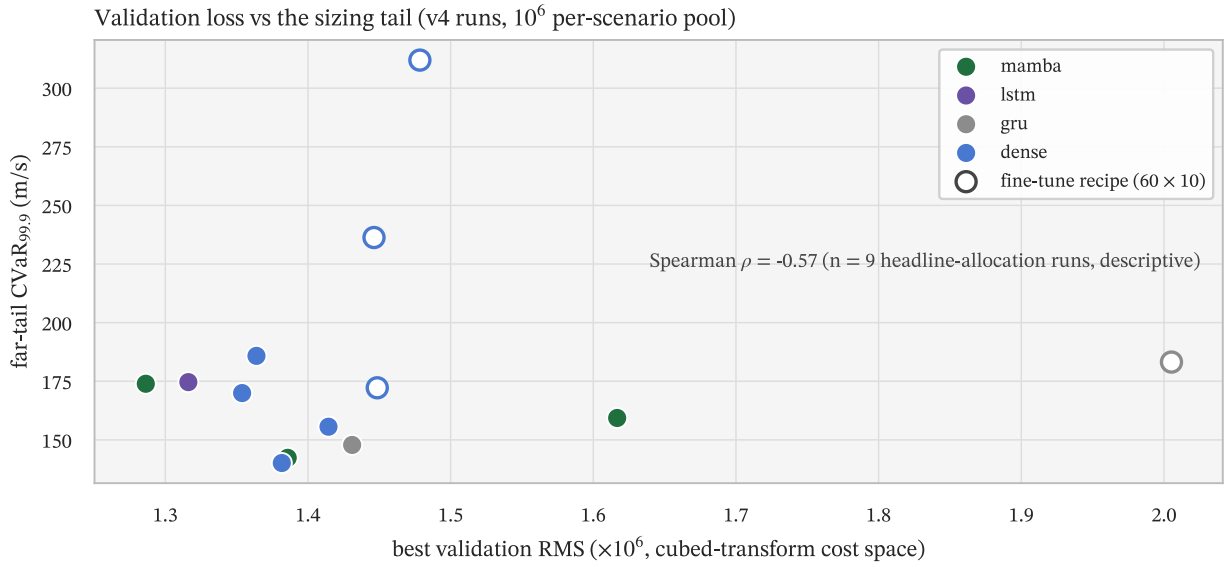


Figure 11: Best validation RMS versus far-tail $\mathrm{CVaR}_{99.9}$ on the confirmatory pool for the nine headline-allocation runs (filled) and four bundled runs of another recipe (hollow: shared-path champions fine-tuned under per-scenario noise, population 60, ten scenarios per individual, 2000 generations). The Spearman rank correlation over the nine same-allocation runs is printed on the figure and read as descriptive (the runs are not exchangeable across families): the loss the gate selects on does not order the sizing tail.

The deployed headline policy is seed 1 of the Mamba cell: a 962-parameter dense-to-selective-state-space-to-dense stack (a 17-input dense encoder, a Mamba core of inner width 16 and state size 12, a two-output dense decoder with the atan2 bank decoder), trained under the full methodology of Section 4 and deployed by the rule stated before any confirmatory score existed. It captures 100% of the time at 111.0 m/s mean and 122.8 m/s CVaR_{95} on a fresh, never-trained-or-selected-on pool, against 110.7 and 119.7 on the final-evaluation pool of Table 7, and on the confirmatory pool captures 10^6 of 10^6 scenarios at $\mathrm{CVaR}_{95} = 120.1$: there is no selection optimism in the deployed seed's shallow tail, and its far tail is the one Table 5 reports. The 515-parameter dense network is the **efficiency reference**: half the parameters, no internal state, a competitive median and, on this evidence, a far tail inside the Mamba's spread; if compute or simplicity is the binding constraint it is the better pick.

6.4 Noise conditioning: a policy with state fits the history it sees

The regime matters more than the architecture, and the development regime shows why. There every scenario of a pool shared one realization of the density perturbation, and the policies trained under it were the headline of an earlier version of this work. Scored on a paired $n = 1000$ pool under both regimes (Table 6), the classical laws lose 11–31 m/s of CVaR_{95} when each scenario receives its own realization – ordinary distribution widening – while the five shared-path-trained networks lose 54–84, 2–4 × more, and every one adds heat-load violations, which no law incurs. A policy with internal state can fit the one density history it ever sees, and did. The nine cells of Table 5, trained under per-scenario noise, score the two regimes within 3.6 m/s of each other: training on the marginal

distribution is what makes a policy's number a marginal number, and it is the one change between that earlier headline and this one.

Table 6: Shared-path versus per-scenario CVaR_{95} (m/s, paired $n = 1000$) for the development-regime champions and the classical laws tuned in that regime. Beside each value, a capture rate below 100% and any heat-load violation are noted, and nothing else is. The networks give up 2–4 × more tail than the classical schemes; on the per-scenario tail the development-regime FNPAG beats every shared-path-trained network.

Shared-path-trained policy	Shared path	Per-scenario	Δ
Mamba 962	115.9	194.4 (98.0% capture, 1.2% heat-load viol.)	+78.5
LSTM 1082	116.3 (14.6% heat-load viol.)	170.4 (17.0% heat-load viol.)	+54.1
GRU 1014	119.3	198.1 (98.0% capture, 1.5% heat-load viol.)	+78.8
Dense 972	121.3	194.9 (99.3% capture, 0.5% heat-load viol.)	+73.5
Dense 515	117.4	201.8 (0.3% heat-load viol.)	+84.3
FNPAG	143.3	154.3 (99.4% capture)	+11.0
PredGuid / FTC	221–231	244–261 (PredGuid 99.9% capture)	+22–+31

A final architectural detail concerns the bank decoder, settled in the development regime on the 515-parameter dense cell of Section 5. Among the single-output decoders that attack the $\pm\pi$ wrap seam, the classical two-output atan2 decoder of Equation 2 still wins: it reaches 117.4 m/s mean and 128.7 m/s CVaR_{95} against the delta decoder (119.9/141.6) and the scaled-π decoder (122.2/140.4), with the edge concentrated on the tail (roughly 12–13 m/s at CVaR_{95}, paired) (Figure 12). The decoder we inherited from 2009 is still the right one.

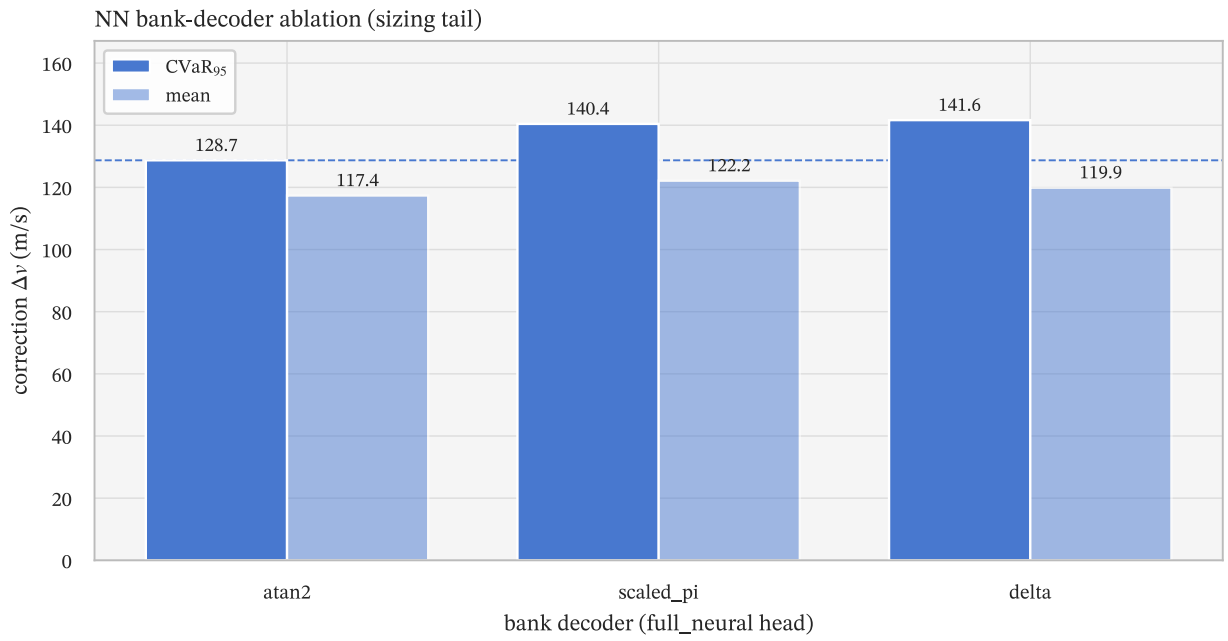


Figure 12: Bank-decoder variants on the 515-parameter dense network, in the development regime. The two-output atan2 decoder inherited from the 2009 work wins, with most of its advantage on the tail.

7 Classical versus neural network

We now place the neural policy against the classical schemes on identical Monte Carlo scenarios – the same seed pools, the same dispersions, the same per-scenario density noise. Two things have to be established: that the classical baselines are tuned to their best, and that the comparison is read at sizing depth.

7.1 A co-optimized reference recovers the predictor–correctors

Before comparing against the classical schemes we must give them their best shot. Every classical scheme was retuned by the same genetic algorithm on the same cost under per-scenario noise (Appendix A). The reference-tracking laws (FTC, the energy controller, PredGuid) are only as good as the reference trajectory they enslave to, and the legacy constant-bank reference is not optimal. We therefore let the genetic algorithm co-optimize the reference: a single extra gene sets the constant bank angle that generates the reference table, regenerated per individual, so the law and the trajectory it tracks adapt together. The effect is large (Figure 13). FTC falls from 175.1 to 126.6 m/s mean and from 244.4 to 146.3 at CVaR_{95} (paired −48.6 and −98.1, Table 8), and at the far tail from 349.7 to 177.8. The energy controller recovers 53 m/s of mean and PredGuid 39. The reference **was** FTC's weakness: a feedback law tracking a poor target cannot out-perform the target.

With its reference repaired, FTC becomes the best classical scheme at sizing depth: it ties FNPAG on the paired pool (+1.9 m/s in mean, CVaR_{95} interval straddling zero) and pulls ahead at the far tail (177.8 against 189.1, Section 7.2) – and it is analytic and fast. The aerocapture story we wrote in 2009, in which FTC was the baseline to beat, becomes: a **well-referenced** FTC is the classical state of the art, and it costs a fraction of a numerical predictor–corrector to run.

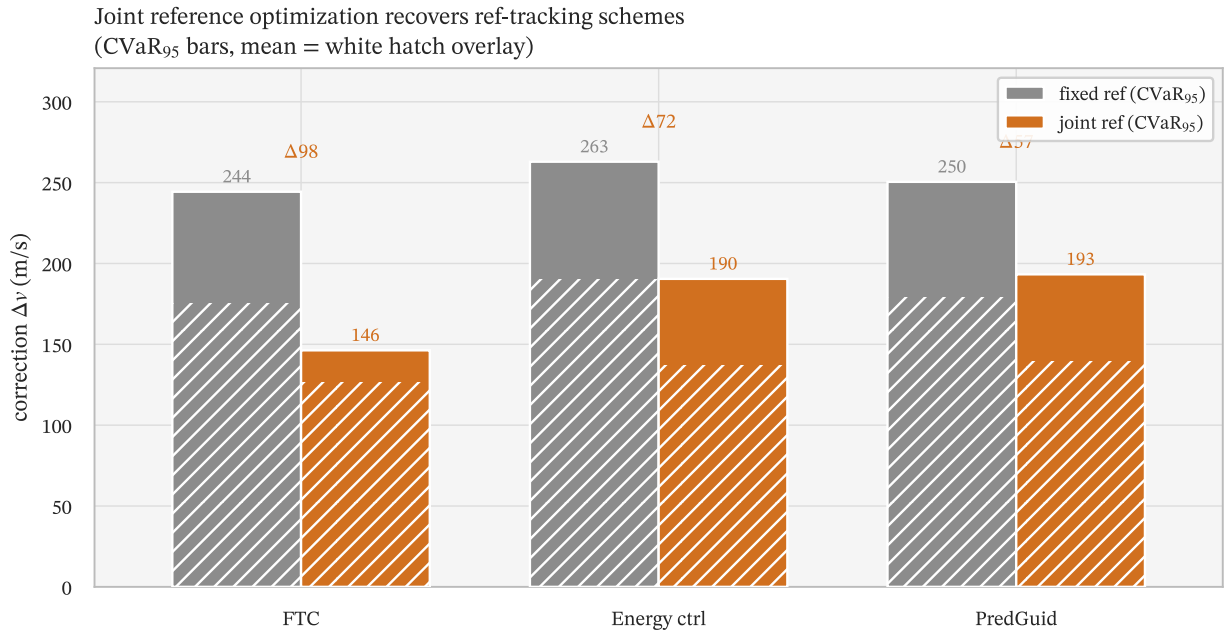


Figure 13: Fixed versus co-optimized reference for the three reference-tracking schemes, retuned under per-scenario noise and scored on the $n = 1000$ final-evaluation pool. Co-optimizing the constant-bank reference recovers most of FTC's deficit and lifts it to the best classical scheme.

7.2 The deployability triangle

Three schemes define the deployability frontier: the neural network, the well-referenced FTC, and FNPAG. They trade off along three axes – accuracy, compute, and robustness – and no single scheme dominates all three.

Accuracy. On the paired $n = 1000$ pool the deployed network wins at every statistic: against joint-FTC by 15.9 m/s in mean, 22.0 at p_{95} and 26.6 at CVaR_{95}, winning 99.4% of scenarios; against FNPAG the margins are 14.0 / 22.1 / 24.5 m/s, winning 99.8% (Table 8). The tail margin is consistently **larger** than the mean margin – the network's advantage is precisely where the mission is sized. At the sizing depth itself the margin narrows to the seed spread of Section 6.2: the deployed seed's $\mathrm{CVaR}_{99.9}$ of 173.9 ± 2.2 sits 3.9 m/s below joint-FTC (177.8 ± 0.5) and 15.2 below FNPAG (189.1 ± 1.8); the Mamba family mean sits 19.3 and 30.6 below. FNPAG's extreme tail fattens with depth in a way its shallow statistics hide – it ties joint-FTC at CVaR_{95} and loses 11.3 m/s at $\mathrm{CVaR}_{99.9}$, with 238 physical crashes in 10^6 (surface impacts on individual re-runs without the evaluation timeout, not censoring) – so the repaired FTC is the better classical scheme at sizing depth.

Compute. On a single idle core, the dense network runs at 1.78 ms per simulation and the stateful Mamba at 3.28 ms, against 1.29 ms for the joint-reference FTC and 207.6 ms for FNPAG (Figure 14). The network is about $2.5\times$ FTC – the same fast class – and $63\times$ faster than the numerical predictor–corrector. On accuracy and compute FNPAG is dominated – joint-FTC matches its accuracy and the network beats it, both at a small fraction of its cost – though the off-nominal stress below keeps robustness a separate axis. The selective-state-space core costs about $1.8\times$ the dense network, the price of a state whose tail benefit Section 6.3 could not resolve. Per control action rather than per trajectory: the network's forward pass costs ≈ 4 µs per 1 s guidance update (whole-simulation cost divided by the ≈ 771 updates flown – an upper bound on pure inference), while FNPAG's predictor work amounts to ≈ 0.65 ms per 2 s replan cycle. Flight processors run one to two orders of magnitude slower than the laptop core measured here; at a conservative $100\times$ scaling neither scheme breaches its own deadline (≈ 65 ms per replan against 2 s; ≈ 0.4 ms per update against 1 s), but the deadline margins differ by a factor of about 76, and the relative cost ordering is host-independent. None of these measurements establish worst-case execution time or memory on qualified hardware (Section 9).

Robustness – the caveat. We trained the network on the medium dispersion regime. Under a deliberately harsher off-nominal regime (atmosphere, density perturbation, navigation, and filter all set high; Appendix A), scored at $n = 10,000$ with bootstrap intervals, the picture inverts, and we report it plainly because it is the one place the network loses (Figure 15). All stress-regime tail statistics are conditional on capture – $\mathrm{CVaR}_{95}(\Delta v \mid \mathrm{capture})$ – and a conditional tail can improve by failing the hardest scenarios, so we read every stress comparison lexicographically: capture probability first (a difference within half a point read as parity), conditional tail cost second, and no tail win is claimed across a larger capture-rate deficit. The deployed network captures 85.3% [84.6, 86.0] of the stress pool, the lowest of the five schemes: 4–11 points below the classical laws, every paired deficit with an interval excluding zero (joint-FTC 95.9%, FNPAG 92.9%, fixed-reference FTC 92.4%, PredGuid 89.3%). Its conditional tail, 658 m/s, is worse than every law's but FNPAG's (468 for joint-FTC, 716 for FNPAG). The lesson is not that the network is fragile – it captures 85% far outside its training regime – but that a training-free analytic law extrapolates better than a policy trained on a narrower distribution. The network wins the nominal sizing tail it was trained for; Section 7.3 asks whether the gap is intrinsic.

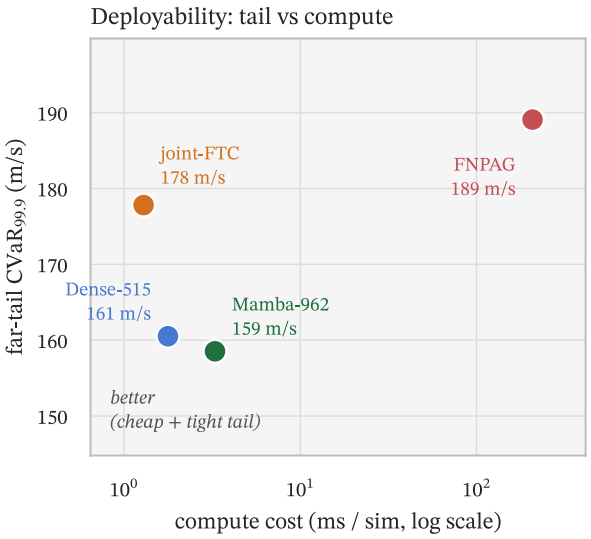


Figure 14: Deployability: per-simulation compute versus far-tail $\mathrm{CVaR}_{99.9}$ on the confirmatory pool (network points are three-seed means). Both network points sit lower-left – a tighter tail than every classical scheme and far cheaper than FNPAG; the seed spread of Table 5 is the width of that margin.

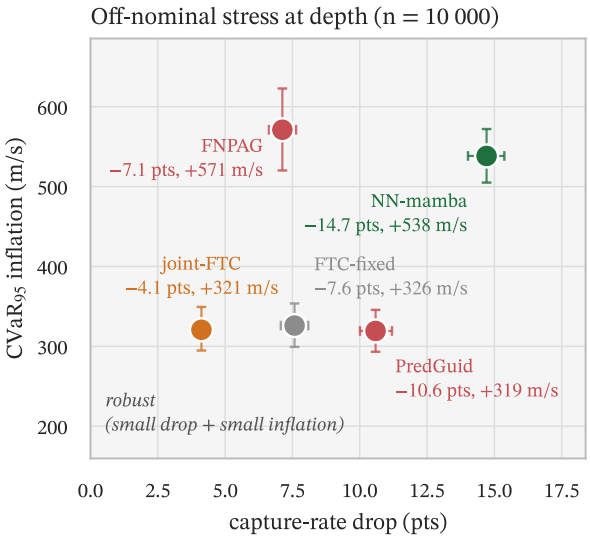


Figure 15: Off-nominal stress at $n = 10,000$: capture-rate drop and CVaR_{95} inflation from each scheme's nominal confirmatory numbers, with bootstrap 95% whiskers. The medium-trained network loses the most capture of the five; the analytic laws extrapolate better.

Table 7: Final Monte Carlo performance under per-scenario noise, correction Δv in m/s, ordered by CVaR_{95}. Capture / mean / p_{95} / CVaR_{95} are on the $n = 1000$ final-evaluation pool; †Viol. and $\mathrm{CVaR}_{99.9}$ are on the frozen confirmatory pool ($10 \times 100,000$ scenarios per scheme; each $\mathrm{CVaR}_{99.9}$ averages 1000 tail observations, replicate standard errors in Table 5). Network rows are the deployed seed of each cell (seed 1, fixed by rule); the Mamba and dense-515 three-seed means are in Table 5. Viol. is the share of confirmatory draws exceeding any of the heat-flux, g-load, or heat-load limits: the networks' exceedances are heat load at the 10^{-5}–10^{-3} level (Section 6.2), the fixed-reference FTC's and the two signed-bank schemes' are heat flux, and the joint-reference trackers and FNPAG exceed nothing or next to it. Every classical scheme was retuned under per-scenario noise at one allocation (Appendix A); ‡these two were selected without the strict feasibility ceiling the other seven met, because no candidate passed it. A 100% capture rate means "no failures observed": with zero failures in n independent scenarios the one-sided 95% upper bound on the failure probability is $\approx 3/n$. The mean is reported for continuity with the 2009 work but is operationally secondary to the tail.

Scheme	**Capture %**	**Viol. %†**	**Mean**	p_{95}	$\mathbf{CVaR}_{95}$	$\mathbf{CVaR}_{99.9}$†
NN – Mamba 962 (deployed)	100.0	0.11	110.7	116.4	**119.7**	**173.9**
NN – GRU 1014	100.0	0.02	113.0	119.4	122.1	147.8
NN – dense 515 (efficiency ref.)	100.0	0.15	113.4	120.6	124.2	140.1
NN – dense 972	100.0	0.27	113.1	120.8	125.7	170.0
NN – LSTM 1082	100.0	0.00	114.7	123.8	130.8	174.6
FNPAG	100.0	0.00	124.6	138.4	144.2	189.1
FTC (joint reference)	100.0	0.00	126.6	138.4	146.3	177.8
Energy controller (joint reference)	100.0	0.02	137.2	168.4	190.5	299.1
PredGuid (joint reference)	100.0	0.01	139.5	172.9	193.3	340.4
FTC (fixed reference)	100.0	0.12	175.1	218.1	244.4	349.7
PredGuid (fixed reference)	99.8	0.01	178.9	222.6	250.5	332.6
Energy controller (fixed reference)	99.4	0.01	190.1	238.1	262.9	340.4
Equilibrium glide‡	99.6	0.54	192.9	292.1	339.6	476.7
Piecewise constant‡	100.0	3.13	265.0	399.4	440.4	632.2

Table 8: Paired comparisons on the $n = 1000$ final-evaluation pool under per-scenario noise, correction Δv in m/s; negative Δ favors A. The intervals are 10,000-resample paired bootstraps. Win-rate and p (Wilcoxon signed-rank) are computed on the per-scenario cost; p is truncated at 10^{-15} – the normal-approximation statistic saturates at or near sign unanimity at $n = 1000$, certifying a (near-)unanimous direction, not a resolved tail probability. Deltas are computed on unrounded per-scenario values, so a delta may differ from the difference of the rounded marginals in Table 7 by 0.1 m/s. ‡For the intra-network rows the deployed seed wins the shallow tail with intervals excluding zero; at sizing depth the families overlap (Table 5).

A vs B	Δmean	95% CI	Δp_{95}	ΔCVaR$_{95}$	95% CI	A-win %	p
Mamba vs FTC, fixed ref.	-64.5	$[-66.1, -62.9]$	-101.7	-124.7	$[-136.3, -113.8]$	100.0	$< 10^{-15}$
Mamba vs FTC, joint ref.	-15.9	$[-16.4, -15.4]$	-22.0	-26.6	$[-30.1, -23.1]$	99.4	$< 10^{-15}$
Mamba vs FNPAG	-14.0	$[-14.4, -13.6]$	-22.1	-24.5	$[-27.1, -21.8]$	99.8	$< 10^{-15}$
Mamba vs dense 515	-2.7	$[-3.0, -2.5]$	-4.3	-4.5	$[-6.3, -2.4]$	79.7‡	6×10^{-89}
Mamba vs GRU 1014	-2.3	$[-2.5, -2.1]$	-3.0	-2.5	$[-3.5, -1.5]$	73.3‡	9×10^{-69}
Mamba vs dense 972	-2.4	$[-2.7, -2.2]$	-4.4	-6.1	$[-8.8, -3.4]$	77.1‡	9×10^{-79}
Mamba vs LSTM 1082	-4.1	$[-4.4, -3.7]$	-7.5	-11.1	$[-15.0, -7.6]$	84.0‡	$< 10^{-15}$
FTC: joint vs fixed ref.	-48.6	$[-50.1, -47.0]$	-79.7	-98.1	$[-109.8, -87.2]$	99.8	$< 10^{-15}$
FTC, joint vs FNPAG	$+1.9$	$[+1.4, +2.5]$	-0.1	$+2.1$	$[-1.5, +5.8]$	39.5	6×10^{-14}

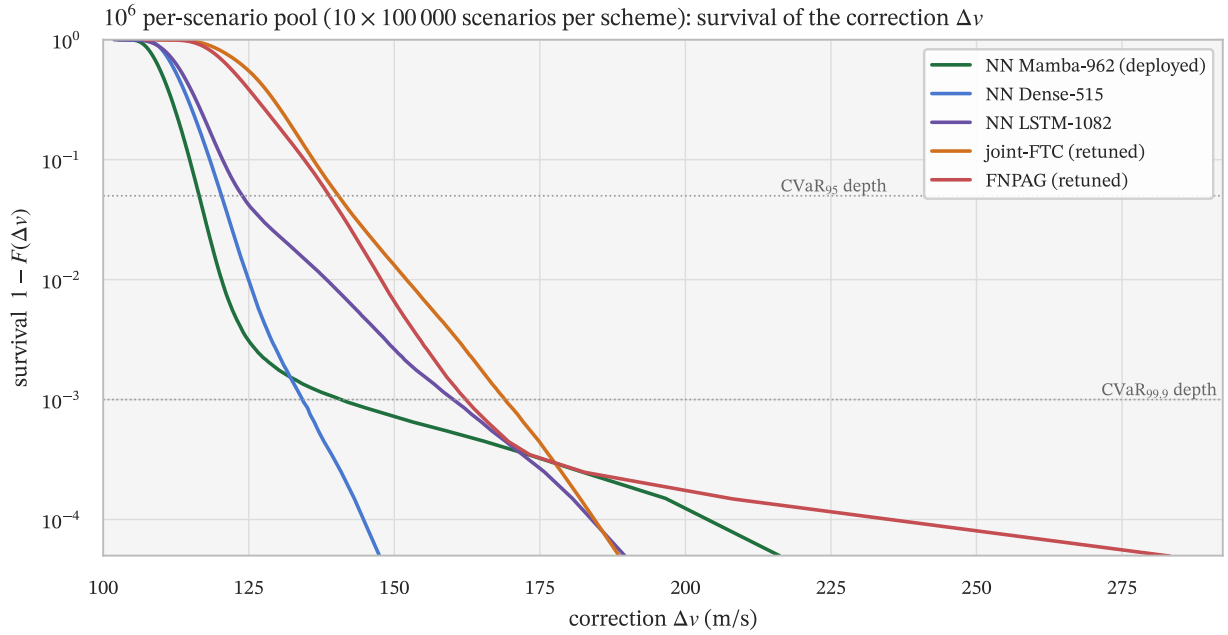


Figure 16: Empirical survival curves of the correction Δv on the confirmatory pool ($10 \times 100,000$ scenarios per scheme, per-scenario noise; log scale, curves subsampled every $\approx$ 100th order statistic) for the deployed Mamba and dense seeds, the LSTM cell, and the two retuned classical references. The network-to-classical separation holds at every depth; the network curves cross one another.

7.3 Matching the objective to the regime

The robustness caveat of Section 7.2 invites a question: is the off-nominal gap intrinsic to neural guidance, or an artifact of a training objective tuned for the wrong regime? Before answering it, the regime itself deserves scrutiny. The high-dispersion stress pool is not a realistic operating point – no mission would be flown on an entry profile that fails roughly one pass in twenty under **any** guidance scheme. In practice the entry interface would simply be

re-targeted so that the 3σ dispersion envelope still captures, restoring a near-certain margin by construction. We therefore read this regime not as a deployment scenario but as a deliberate **stress probe of the training method**: it pushes the optimizer into a regime where roughly 5% of scenarios are catastrophic, and asks whether the genetic algorithm can still make progress.

Under that probe, the deployed objective cannot. Training on the high regime with the cubed transform, the hardest-seed curation and two scenarios per individual (the **stacked** cell of Figure 17) reaches 93.4% capture with a CVaR_{95} of 1252 m/s on the stress pool, while exceeding the heat-flux limit on 33% of draws. The mechanism is the one the medium regime hides: when failures dominate, cubing the cost makes the objective a spiky near-discrete failure count, max-bucket curation feeds the optimizer only the hardest seeds, and a per-individual batch of two cannot estimate any of it – so selection has no gradient to climb. The worst-case shaping that is a virtue in the medium regime becomes a liability once the noise outruns the sample budget.

Centering the objective – more scenarios per individual (a real cost estimate), a central curation bucket, and a milder transform – restores the gradient. On the dense vehicle a one-lever-at-a-time sweep (Table 9, Figure 17) attributes the recovery, with the constraint exceedance read beside every tail number because the levers trade them. Every lever lowers the stacked tail; the middle bucket does the most alone (CVaR_{95} 1252 $\rightarrow$ 462) and carries the worst heat-load exceedance of the five (8.1%); sixteen scenarios per individual add 1.7 capture points and leave the heat-flux exceedance where the stacked objective put it (31% of draws), the linear transform adds 1.8 points and cuts it to 9.4%; only all three together reach the lowest tail (357 m/s) with every exceedance low (9.2 / 1.0 / 1.3% for heat flux, g-load and heat load). At $n = 1000$ a capture rate carries about 1.4 points of binomial half-width, so the capture column is read as a direction, and the centered and middle-bucket tails overlap. The three knobs are a coupled system, exactly as in the medium regime, but now the coupling has teeth.

Table 9: The objective levers on the high-regime stress pool ($n = 1000$, per-scenario noise), one at a time from the stacked objective: capture, mean and CVaR_{95} of the correction Δv over captured scenarios (bootstrap 95% interval in brackets), and the share of draws exceeding each constraint limit.

Dense 515 cell	Capture (%)	Mean	CVaR_{95} (m/s)	Heat flux (%)	g-load (%)	Heat load (%)
Stacked: cubed, max bucket, $n = 2$	93.4	601	1252 [1200, 1292]	33.4	13.2	0.0
+ 16 scenarios per individual	95.1	417	840 [792, 896]	31.0	14.9	0.0
+ middle bucket	93.5	151	462 [342, 594]	3.0	0.7	8.1
+ linear transform	95.2	230	634 [568, 703]	9.4	1.3	2.8
Centered: all three	95.3	157	357 [276, 458]	9.2	1.0	1.3

The effect transfers to the deployed architecture, and the comparison is read at sizing depth: three independent centered-Mamba retrainings and two joint-FTC baselines – the medium-deployed retune of Table 7 and a joint-FTC retuned on the high regime – scored on $n = 10,000$ scenarios of the same stress pool, paired on scenario, with bootstrap 95% intervals (Table 10). The three seeds hold capture at 96.26–96.34% against 95.9% for the medium-deployed joint-FTC, within the half-point parity band, and every seed wins the conditional tail: $\mathrm{CVaR}_{95}(\Delta v \mid \text{capture})$ of 334–375 m/s against 468, every paired delta negative with an interval excluding zero. Against the high-regime retune (92.8%, 713 m/s) every seed wins both capture and tail, but that baseline is degraded – no candidate of its run was gate-feasible and it loses to the medium retune on the same pool – so the medium comparison is the binding one. On that evidence the off-nominal gap is not intrinsic to neural guidance: the analytic law's edge in Section 7.2 was a property of the **mismatched** training objective.

The price is stated in the same table. No policy of either family trained on the high regime passes the strict feasibility gate; the comparison stands at the exceedance each policy shows, and the centered seeds exceed the heat-flux limit on 8.5–18.1% of draws against 11.1% for the medium joint-FTC, and the heat-load limit on 1.4–2.1% against 0.5%. Seed 1 pays for part of its tail in heat flux; seed 3 wins its tail with less heat-flux exceedance than the baseline; across

the three seeds the tail ranking is the reverse of the heat-flux ranking, which three points make an observation, not a measured trade-off. The stress regime itself remains one a real mission would design away. The durable lesson is methodological and reinforces Section 4: the optimal worst-case weighting is matched to the environment's noise and the per-individual sample budget, not fixed once – and the constraint exceedance is part of what the match has to hold.

Table 10: The centered high-regime cells at sizing depth: $n = 10,000$ scenarios of the 9M stress pool under per-scenario noise; capture and the conditional CVaR_{95} of the correction Δv with bootstrap 95% intervals, the share of draws exceeding each constraint limit, and each seed's paired CVaR_{95} delta against the medium-deployed joint-FTC (negative: the seed's tail is better). Every seed sits at capture parity with the medium baseline and beats its tail; no cell of either family is feasible under the strict gate.

Cell	Capture (%)	CVaR_{95} (m/s)	Heat flux (%)	g-load (%)	Heat load (%)	Δ vs medium
Centered Mamba, s1	96.3 [95.9, 96.6]	334 [305, 364]	18.1	2.4	2.1	-135 [−157, −112]
Centered Mamba, s2	96.3 [96.0, 96.7]	352 [323, 382]	12.2	1.6	1.4	-116 [−138, −93]
Centered Mamba, s3	96.3 [95.9, 96.6]	375 [346, 406]	8.5	1.1	1.5	-94 [−116, −69]
Joint-FTC, medium	95.9 [95.5, 96.3]	468 [442, 497]	11.1	1.8	0.5	–
Joint-FTC, high regime	92.8 [92.3, 93.3]	713 [684, 741]	10.6	1.5	0.0	–

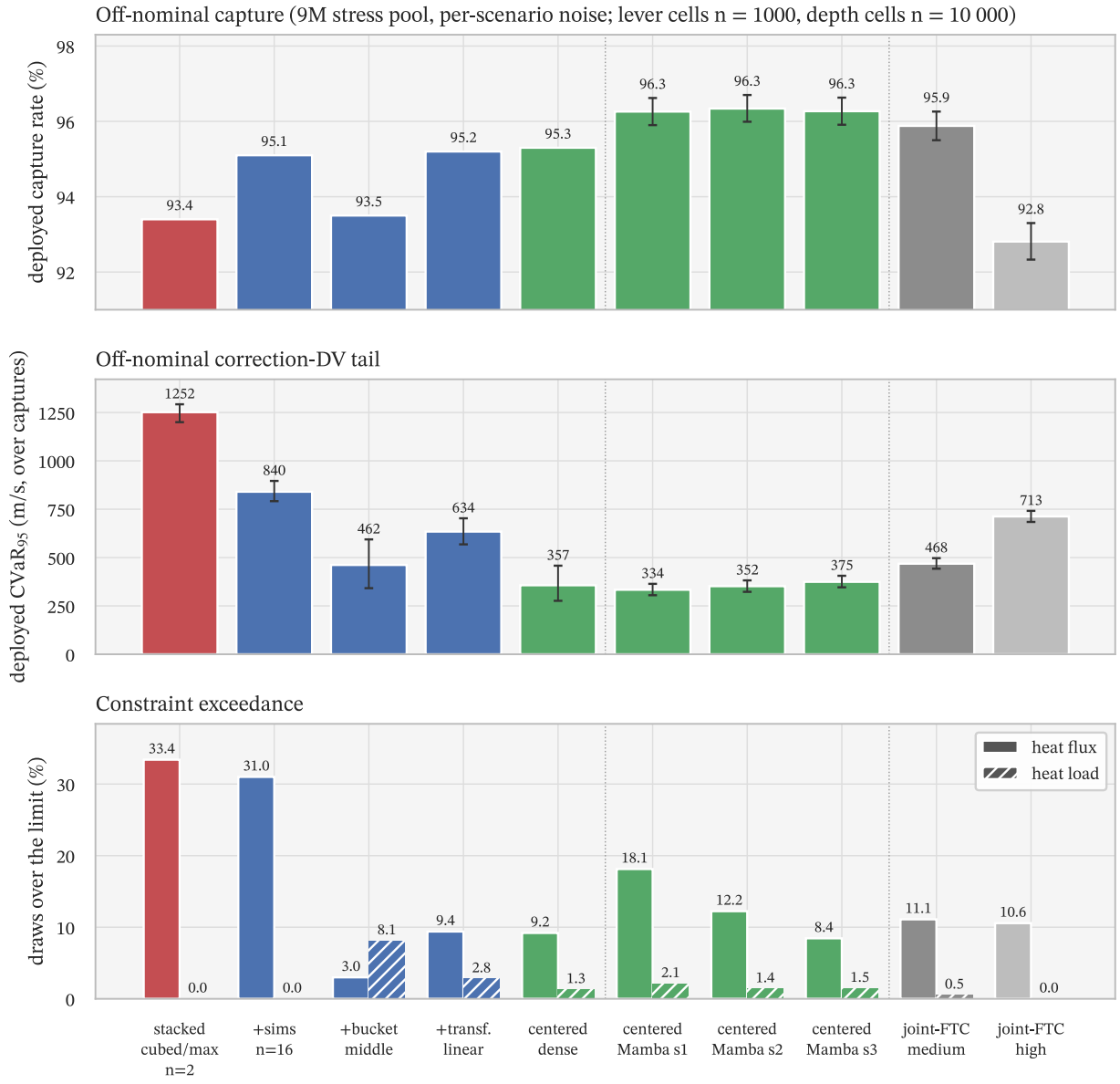


Figure 17: Objective centering under the high-dispersion stress probe (9M pool, per-scenario noise), three panels over the same ten cells: capture (top), conditional CVaR_{95} (middle), and the heat-flux (solid) and heat-load (hatched) exceedance shares (bottom). The five dense lever cells are the $n = 1000$ rows of Table 9; the three centered-Mamba seeds and the two joint-FTC baselines are the $n = 10,000$ rows of Table 10, with bootstrap 95% whiskers. Centering recovers the tail and transfers to the Mamba; the exceedance panel is read with the tail panel, because the levers and the seeds trade them.

8 What the network uses

To close the architecture argument we ask which inputs the deployed Mamba policy actually relies on, through a closed-loop input-sensitivity analysis: each input is zeroed in turn and the resulting cost increase measured on $n = 1000$ scenarios under per-scenario noise (Figure 18). Zeroing a normalized input perturbs every subsequent state of the closed loop, so the deltas measure closed-loop dependence, not isolated feature importance. All seventeen masked

inputs cost something when zeroed. The largest degradation by far comes from the orbital energy – the coordinate of the corridor itself – and immediately behind it two of the engineered, cost-aligned **predicted correction-Δv** components: the energy-closing burn (`predicted_dv1`) and the periapsis-correction burn (`predicted_dv2`), each evaluated on the current osculating orbit. The reference-trajectory altitude rate, the radial velocity and the dynamic-pressure error – the quantities a reference-tracking law would feed back on – follow; the four bank-history encodings and the eccentricity excess contribute least.

This is the flat median of Section 6 seen from the inside. The network's strongest dependencies after the energy are signals that tell it, causally and smoothly, what the maneuver would cost if it stopped now – a cost-to-go surrogate handed to the policy as an input. Because the osculating orbit is itself an integral of the flown trajectory, those signals already summarize most of the history a memory cell could recover, and a dense network without any memory matches the recurrent ones on the median: the engineered inputs do the work that recurrence would otherwise do. The no-predicted-Δv control of Section 6.3 is the same finding at the tail: retrained without these three inputs the policy reaches the same far tail within seed variance, so the inputs are a convenience the search exploits rather than information the policy cannot recover from the rest of the state. The network is using the physics we handed it; whether it reserves its memory for the tail is the question the controls left open.

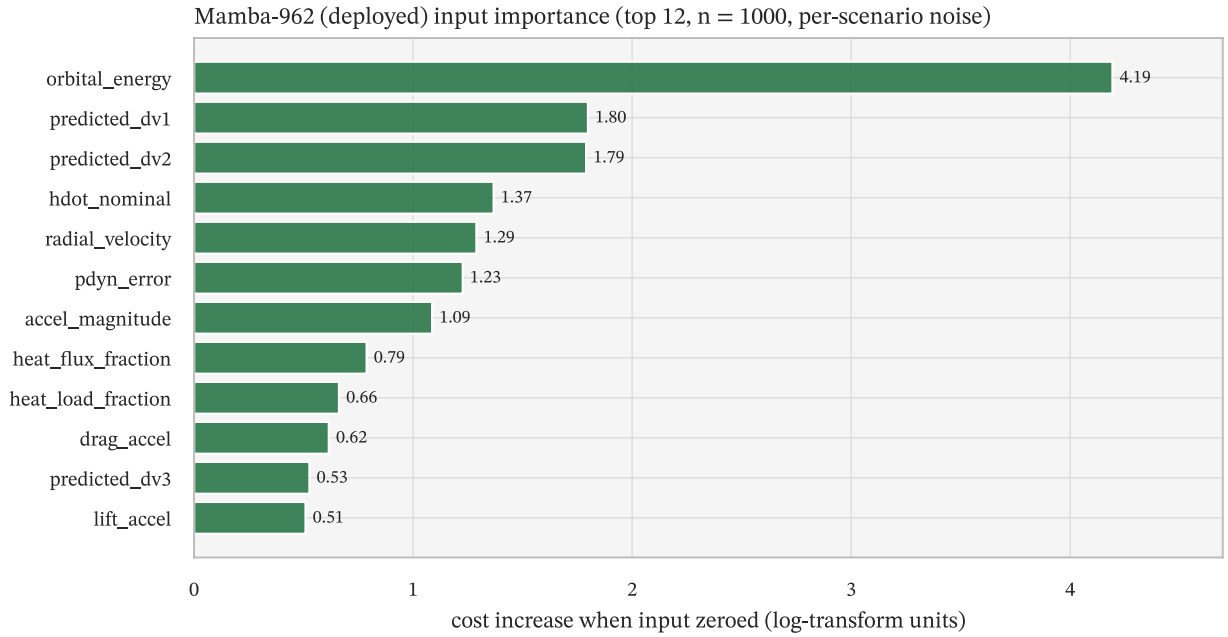


Figure 18: Closed-loop input sensitivity: per-input cost increase when each input is zeroed, for the deployed Mamba policy ($n = 1000$, per-scenario noise, log cost transform). The orbital energy dominates, followed by the engineered, cost-aligned predicted-Δv components and the reference-tracking signals.

9 Discussion and limitations

The clearest limitation is the off-nominal robustness gap of Section 7.2 – the deployed network wins the nominal sizing tail it was trained for and loses, off-nominal, to a training-free analytic law, capturing 4–11 points less than the classical schemes under the stress regime. Section 7.3 traces that gap to the training objective rather than to neural guidance: at sizing depth the centered retrain beats the medium-deployed joint-FTC on the conditional tail at capture parity, across three seeds, but it buys part of that tail with heat-flux exceedance, no high-regime policy of either family is feasible under the strict gate, and the stress regime is one a real mission would design away. What remains open is a centered retrain whose constraint exceedance matches the analytic law's.

The second limitation is the one the per-scenario campaign measured on our own earlier claim. The networks beat the classical schemes at sizing depth by a margin the seed spread does not erase, but they beat one another by a margin it does: the Mamba's three seeds span 31.6 m/s of $\mathrm{CVaR}_{99.9}$ and the dense reference's 45.7, the controls without learned state or without the cost-to-go inputs are not resolved from the Mamba family (Section 6.3), and the deployed seed is the worst of its family at the depth that sizes the tanks. A sizing engineer should quote a family mean with its spread, repeat seeds before quoting any architecture ordering, and expect the gate – which promotes on validation loss – to pick a seed the far tail does not favor. The broader pattern, seen three times – the shared density history of Section 6.4, the off-nominal regime, the heat-load ceiling a policy carries – is that the network is exactly as good as the distribution it trains on, and widening the training environment recovers what the narrow one gave away.

A third tradeoff is the cost of state. The deployed Mamba runs at 3.28 ms per simulation against 1.78 ms for the dense network – about $1.8\times$ for the selective-state-space core. The head itself accounts for ≈ 1.5 ms of that cost; single-precision arithmetic would cut it by about a third, while integer quantization buys memory rather than speed at this scale (Appendix C). Both remain in the fast compute class, an order of magnitude below the numerical predictor–corrector, so the choice is between the network and FTC, not between the network and FNPAG. If on-board compute or implementation simplicity is the binding constraint, the memoryless 515-parameter dense network is the efficiency reference: a competitive median at half the parameters, and a far tail this campaign cannot separate from the recurrent one.

A flight-qualification path for a stateful network remains open, and the deployability triangle suggests its shape: a simplex arrangement in which the analytic joint-FTC – whose off-nominal robustness Section 7.2 established – runs in parallel as an onboard monitor with authority to take over on envelope violation, while the network flies the nominal corridor it wins on. The deployed policy is a fixed forward pass of 962 parameters in double precision with no data-dependent control flow, so worst-case execution time and memory bound trivially; verifying the **decision** behavior across the dispersion envelope, rather than the code, is the open problem. Appendix C quantizes the deployed head (weight-only): 8-bit is free, and a 4-bit projections-only head reached by quantization-aware fine-tuning holds the sizing tail at a $5.7\times$ memory reduction – pruning remains open. Run-to-run variance is calibrated at the tail through the three-seed repeats of every deciding cell and, for the decisive seed-strategy and CMA-ES cells of the development regime, at the mean through dedicated repeats (Section 4.1); the tight optimizer differences of Section 5 stay reported as indistinguishable rather than ranked.

Finally, a methodological note for anyone reproducing this. The training is not bit-reproducible from a seed alone – it never was – because the non-stationary objective and the operator randomness make each run a fresh draw; the deployed policy is reproduced from its saved weights and checkpoints, not re-derived. This is the same property that forced us to measure run-to-run variance directly rather than assume it away.

10 Conclusion

Seventeen years ago we showed that a feed-forward network trained by a genetic algorithm could fly an MSR aerocapture more efficiently than a Cerimele–Gamble feedback law, and we asked for a comparison against predictor–correctors. This paper delivers it, and the answer is favorable to neural guidance on the metric that matters, with its limit measured. Under independent per-scenario density noise, on 10^6 pre-registered confirmatory scenarios, a 962-parameter recurrent (Mamba) policy trained from scratch holds a far-tail $\mathrm{CVaR}_{99.9}$ of 158.5 ± 15.8 m/s over three training seeds, 19 m/s below a well-referenced FTC and 31 below the numerical predictor–corrector, which it out-runs $63\times$; on the paired pool it beats both at every statistic on more than 99% of scenarios. A memoryless 515-parameter dense network lands inside that seed spread, and so do the matched controls without learned state or without the engineered cost-to-go inputs: the networks beat the classical schemes by a margin the spread does not erase, and beat one another by a margin it does.

Two findings carry beyond the headline number. The first is methodological: a genetic algorithm is the wrong optimizer for a fixed objective and the right one for a moving one, and the moving Monte Carlo environment – adaptive seeds, a tail-weighting cost transform, and hardest-case curation – is a matched system that converts it from the worst optimizer we tested (154 m/s three-seed mean) to the best (120); and the noise regime the policy trains on

is part of the match, because a policy with state fits the one density history it sees. The second is about what a policy has to be: engineered, cost-aligned inputs flatten the typical cost across every cell type we tried, the deployed policy computes with its state, and whether that state is needed for the tail is a question three seeds could not answer at this budget. Training loss did not pick the tail winner; the selection gate promotes on it, so the deployed seed is fixed by rule and quoted with its family's spread.

The honest drawback that closed the 2009 paper – that the training is too heavy to run on board – remains, but its sting is gone: the deployed policy is a fixed forward pass that costs a few milliseconds, and the heavy optimization happens once, on the ground – ground compute buying in-flight margin. The next step would be to widen the training environment – annealing the tail-weighting and the dispersion envelope over the run, or stratifying the curated batch across cost quantiles – until the network's off-nominal robustness matches its nominal accuracy at the analytic law's constraint margin, and to carry these stateful policies beyond the single capture maneuver – to skip-entry and Earth-return legs, and to on-line adaptation of the deployed policy in flight.

References

Bairstow, S.H. (2006) *Reentry Guidance with Extended Range Capability for Low L/D Spacecraft*. Master's thesis.

Beck, M. *et al.* (2024) "xLSTM: Extended Long Short-Term Memory," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Calkins, G.E. *et al.* (2025) "Risk-Aware Aerocapture Guidance Through a Probabilistic Indicator Function."

Cerimele, C.J. and Gamble, J.D. (1985) "A Simplified Guidance Algorithm for Lifting Aeroassist Orbital Transfer Vehicles," in *AIAA 23rd Aerospace Sciences Meeting*.

Cho, K. *et al.* (2014) "Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734.

Forget, F. *et al.* (1999) "Improved General Circulation Models of the Martian Atmosphere from the Surface to Above 80 km," *Journal of Geophysical Research: Planets*, 104(E10), pp. 24155–24175.

Gelly, G. and Ferreira, E. (2007) "Guidance Algorithm Using Neural Networks for a Soft-Landing Application," in *European Conference for Aerospace Sciences (EUCASS)*.

Gelly, G. and Gauvain, J.-L. (2015) "Minimum Word Error Training of RNN-based Voice Activity Detection," in *Interspeech*, pp. 2650–2654.

Gelly, G. and Gauvain, J.-L. (2017) "Spoken Language Identification Using LSTM-Based Angular Proximity," in *Interspeech*, pp. 2566–2570.

Gelly, G. and Vernis, P. (2009) "Neural Networks as a Guidance Solution for Soft-Landing and Aerocapture," in *AIAA Guidance, Navigation, and Control Conference*. Chicago, Illinois.

Gelly, G. *et al.* (2016) "A Divide-and-Conquer Approach for Language Identification Based on Recurrent Neural Networks," in *Interspeech*, pp. 3231–3235.

Goldberg, D.E. (1989) *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

Gu, A. and Dao, T. (2023) "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752* [Preprint].

Haarnoja, T. *et al.* (2018) "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in *International Conference on Machine Learning (ICML)*, pp. 1861–1870.

Hansen, N. and Ostermeier, A. (2001) "Completely Derandomized Self-Adaptation in Evolution Strategies," *Evolutionary Computation*, 9(2), pp. 159–195.

Harpold, J.C. and Graves, C.A. (1979) "Shuttle Entry Guidance," *Journal of the Astronautical Sciences*, 27(3), pp. 239–268.

Hasani, R. *et al.* (2022) "Closed-form Continuous-time Neural Networks," *Nature Machine Intelligence*, 4, pp. 992–1003.

Hochreiter, S. and Schmidhuber, J. (1997) "Long Short-Term Memory," *Neural Computation*, 9(8), pp. 1735–1780.

Kennedy, J. and Eberhart, R. (1995) "Particle Swarm Optimization," in *IEEE International Conference on Neural Networks*, pp. 1942–1948.

Lahoti, A. *et al.* (2026) "Mamba-3: Improved Sequence Modeling using State Space Principles," *arXiv preprint arXiv:2603.15569* [Preprint].

Lu, P. (2014) "Entry Guidance: A Unified Method," *Journal of Guidance, Control, and Dynamics*, 37(3), pp. 713–728.

Lu, P. *et al.* (2015) "Optimal Aerocapture Guidance," *Journal of Guidance, Control, and Dynamics*, 38(4), pp. 553–565.

Matz, D.A. *et al.* (2017) "Application of a Fully Numerical Guidance to Mars Aerocapture," in *AIAA Guidance, Navigation, and Control Conference*.

McKay, M.D., Beckman, R.J. and Conover, W.J. (1979) "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code," *Technometrics*, 21(2), pp. 239–245.

Rataczak, J.A., McMahon, J.W. and Boyd, I.D. (2025) "Convex Predictor–Corrector Aerocapture Guidance," *Journal of Guidance, Control, and Dynamics* [Preprint].

Rockafellar, R.T. and Uryasev, S. (2000) "Optimization of Conditional Value-at-Risk," *Journal of Risk*, 2(3), pp. 21–41.

Schulman, J. *et al.* (2017) "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347* [Preprint].

Sonandres, K.A., Palazzo, T.R. and How, J.P. (2025a) "ABAMGuid+: An Enhanced Aerocapture Guidance Framework Using Augmented Bank Angle Modulation."

Sonandres, K.A., Palazzo, T.R. and How, J.P. (2025b) "Real-Time Density Estimation for Uranus Aerocapture Using Deep Learning," in *AIAA SciTech Forum*.

Storn, R. and Price, K. (1997) "Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces," *Journal of Global Optimization*, 11(4), pp. 341–359.

Sun, J., Feng, B. and Xu, W. (2004) "Particle Swarm Optimization with Particles Having Quantum Behavior," in *IEEE Congress on Evolutionary Computation (CEC)*, pp. 325–331.

Vaswani, A. *et al.* (2017) "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30 (NIPS)*, pp. 5998–6008.

Vernis, P. *et al.* (2004) "Guidance Trade-Off for Aerocapture Missions," in *ESA Guidance, Navigation and Control Conference*.

Zucchelli, E.M. *et al.* (2021) "Two Stage Optimization for Aerocapture Guidance," in *AIAA SciTech Forum*.

Appendix A: reproduction details

One compact reference for the settings and systems behind every number.

Simulator. All schemes fly through one native (Rust) simulator: fixed-step fourth-order Runge–Kutta integration (Gill variant), J_2–J_4 zonal gravity, a tabulated Mars atmosphere carrying the static Monte Carlo bias and the Ornstein–Uhlenbeck perturbation, pilot dynamics, thermal tracking, and the navigation–guidance–control chain sequenced on its own cadences; the bias-filter navigation recovers density through lift-corrected inverse dynamics. The implementation is regression-validated against the 2009 study's legacy code – across all 725 time steps of a guided trajectory, 22 of 24 output channels are bit-identical (the two mismatches trace to uninitialized variables in the reference) – which establishes equivalence with the flight-heritage implementation, not physical validation. Independent physics checks run in the test suite: an analytic potential-gradient oracle for the J_2–J_4 gravity expansion, integrator convergence on analytic systems, vacuum energy and angular-momentum conservation, and cross-language forward-pass parity for the network runtime at machine epsilon. The simulator also carries EKF navigation, an altitude-dependent wind model, and adaptive integration; this campaign does not exercise them, so every run here uses the bias-filter navigation and fixed-step integration the validation covers.

Seed pools. Every Monte Carlo pool derives from the mission base seed through disjoint reserved streams: the selection pool (offset 10^6, $n = 1000$, the in-training promotion gate – queried 13037 times over the deployed run, so it is adaptively reused and is not an unbiased estimate of generalization), final evaluation (offset 2×10^6: the $n = 1000$ paired pool of Table 7 and Table 8, and, in the development regime, the $n = 10,000$ far-tail pool of Section 4), the fresh re-quote pool (offset 8×10^6), the off-nominal stress pool (offset 9×10^6), and the architecture-probe pool (offset 10^7, Appendix B). Training scenarios are drawn outside every reserved stream; evaluation pools draw one scenario per seed (independent samples), and multi-scenario batches use Latin-hypercube draws. Table 3 states which decisions each pool influenced and the noise regime each was run in.

Noise seeding. A scenario's density-perturbation stream is seeded from a hash of its own dispersion draw (`noise_seeding = "per_draw"`, the simulator's default): identical draw, identical realization – runs stay reproducible – and distinct scenarios receive distinct paths, so every one-scenario-per-seed pool marginalizes over the process. The development regime seeds the stream from a constant plus the within-batch scenario index, which is zero for every one-scenario batch, so all scenarios of such a pool share one realization. The repository's `arxiv-v2` and `arxiv-v3` drafts of this paper quoted the development regime in their main body; this version is per-scenario throughout, and Section 6.4 is the only place the two are set side by side.

Confirmatory sizing pool. All methodology, architecture, and checkpoint choices were frozen at a recorded revision before the pool was generated, and the deployed seed of every cell (seed 1) was fixed by rule before any cell was scored on it. Seeds are drawn without duplicates from $[2^{31}, 2^{32})$ – structurally disjoint from every earlier draw, since all earlier pools, training batches, and curation probes live in $[0, 2^{31})$ – arranged as ten replicate pools of $100,000$ scenarios sharing seeds across schemes (so per-replicate differences are paired). Every cell is evaluated on it exactly once; tail statistics use the estimator of Section 2.2 ($\mathrm{CVaR}_{99.9}$ averages 100 observations per replicate, 1000 pooled), and replicate-level dispersion gives the quoted standard errors and difference intervals with no distributional assumptions. Non-captures are re-flown individually without the evaluation wall clock and classified: every non-capture of a trained or retuned cell quoted in this paper is a physical crash (the reset-state control of Section 6.3 also logs a handful of timeouts at the simulation's own time limit).

Off-nominal stress regime. The Section 7 stress pool raises four domains to their high presets: the atmospheric density bias to $\pm 100\%$ (from $\pm 50\%$); the Gauss–Markov density perturbation to a 30 s correlation time at 20% RMS (from 120 s at 5%); navigation errors to 3σ altitude ± 3 km, horizontal $\approx \pm 18$ km per axis, velocity ± 3 m/s, drag acceleration ± 0.6 m/s²; and the navigation-filter gain draw to ± 0.45 absolute (3σ).

Dispersion rationale. The $\pm 50\%$ nominal density span is a deliberately conservative envelope for Mars seasonal and dust-loading variability about a tabulated mean profile (Forget *et al.*, 1999); domains are drawn independently as a modeling choice (no cross-correlations are claimed). The static bias and the Ornstein–Uhlenbeck process model different frequencies – profile-scale uncertainty versus along-track variability – and do not double-count; the process is zero-initialized at entry and reaches its stationary variance within roughly one correlation time. The two wind

draw dimensions are retained inert to keep the 26-dimensional draw layout stable across configurations with and without the wind model.

Classical tuning parity. Every classical scheme was tuned by the same genetic algorithm on the same cost C below – identical penalties, virtual costs, and transform – under per-scenario noise, co-optimizing its full parameter set including the shared navigation-filter and actuator gains the network also tunes (26 parameters for FTC against the network's weights plus 3 actuator-side parameters), at 2000 generations $\times$ 300 individuals $\times$ 10 scenarios per generation, one allocation for all nine. Seven schemes were selected under the strict feasibility ceiling of the training gate (zero constraint violations on the selection pool); for the equilibrium-glide and piecewise-constant schemes no candidate passed it in 2000 generations, so their quoted cells were selected without the ceiling and carry their exceedance in Table 7. The classical searches plateau far inside their budget, so the network's longer run buys the comparison nothing. The joint-reference co-optimization of Section 7.1 is the classical counterpart of the network's co-tuned actuator parameters, and the information asymmetry is tested directly: retrained without the three predicted-Δv observations the classical schemes cannot consume, the network reaches the same far tail within seed variance and still beats the co-tuned joint-FTC at CVaR_{95} on every seed and at $\mathrm{CVaR}_{99.9}$ on its three-seed mean, at the heat-load exceedance every network carries (Section 6.3, third control).

Correction burns. The reported Δv is the three-burn plan from the captured orbit (apsis radii r_a, r_p) to the 500 km circular parking orbit (radius r_c), using the elliptical apsis speed $v(r_1, r_2) = \sqrt{2\mu r_2/(r_1(r_1 + r_2))}$: a periapsis correction at apoapsis, $\Delta v_1 = v(r_a, r_c) - v(r_a, r_p)$; circularization at the new periapsis, $\Delta v_2 = \sqrt{\mu/r_c} - v(r_c, r_a)$; and a plane change $\Delta v_3 = 2v_n \sin(\Delta i/2)$ at the cheaper node, v_n the smaller of the target-orbit speeds at the two nodes. The total is $|\Delta v_1| + |\Delta v_2| + |\Delta v_3|$.

Cost and objective. Per simulation the cost is

$$C = \Delta v + s(\Delta v - T) + \frac{s(\Delta v - T)^2}{2S} + \sum_j w_j \, s\big((x_j - L_j)/L_j\big), \tag{7}$$

with s a softplus (a C^∞ knee), $T = 1000$ m/s, S the quadratic scale, and the penalty sum over the peak heat-flux, peak-load, and integrated heat-load exceedances normalized by their limits L_j, weights $w_j = 1$. Non-captures receive a virtual cost above any captured correction: $10,000 + v_\infty$ m/s for hyperbolic escape (v_∞ the excess speed), and $3000 + 1000 \min\big(|E - E_{\mathrm{target}}|, 50\big) - 500\, t/t_{\max}$ for crash and timeout outcomes (energies in MJ/kg) – so captures are always preferred while the optimizer keeps a gradient toward the capture boundary. The per-simulation cost is cubed (the deployed transform) and aggregated across an individual's scenario batch by root-mean-square – the L_6-norm-equivalent objective of Equation 6.

Optimizer (deployed headline). pymoo genetic algorithm (SBX crossover $\eta = 3$, polynomial mutation $\eta = 5$ at rate 0.15), population 512, two scenarios per individual per generation, run for $20,000$ generations. The adaptive seed curation, precisely: every second generation or on a promotion, (1) draw 1000 fresh probe scenarios outside every reserved stream; (2) score them with the current best individual; (3) sort the per-scenario costs into n_{sims} equal-count quantile bins; (4) take the hardest scenario of each bin as the next training batch – scenarios are replaced as a set, never accumulated. The selection gate re-runs each new argmin on the reserved $n = 1000$ selection pool and promotes on strict RMS improvement.

Training harness. Population evaluation is batched through in-process Python bindings to the native core: each generation's individuals are simulated scenario-parallel across CPU cores with the interpreter lock released, network weights pass in memory rather than through files, and the dispersion-independent tables (atmosphere, winds, reference trajectories) are shared read-only across the batch. At the measured single-core cost of Section 7.2, the deployed run's 2×10^7 training passes (5×10^7 with the selection, curation and re-evaluation passes) amount to roughly 49 core-hours – one overnight run on a laptop.

Deployed architecture. Dense($17 \to 16$, swish) $\to$ Mamba($d_{\mathrm{inner}} = 16$, $d_{\mathrm{state}} = 12$) $\to$ Dense($16 \to 2$, asinh), atan2 bank decoder; 962 trainable parameters. The input mask selects 17 of the 35 candidate observations (indices 0, 2, 3, 5, 6, 7, 11, 12, 18, 19, 27–30, 32–34: instantaneous orbital/aerodynamic state, two reference-trajectory interpolations, the seam-free bank-history pairs, and the three predicted correction-Δv components). Each input is normalized by a calibrated per-input affine or asinh transform mapping its $[p_5, p_{95}]$ span to $[-1, 1]$. Three actuator-side parameters –

the navigation density-filter gain and its rate limit, and the command-shaping acceleration limit – are co-optimized with the network weights.

FNPAG. Two-second replan cycle; bisection corrector on the constant capture-bank magnitude (about eleven forward integrations per replan); forward-predictor integration step GA-tuned at 2.9 s; the onboard atmosphere is scaled by the navigation-estimated density factor so the predictor tracks the measured atmosphere.

One runtime, training to flight. The policy that flies is the artifact that trained: candidates are evaluated, selected, and deployed through the same native forward pass, so no export or re-implementation step separates the training loop from the flight code. An independent PyTorch implementation of every cell type (used by the supervised warm-start path, not by the deployed runs) is held to numerical agreement with the native runtime by cross-language tests – maximum absolute forward-pass difference near machine epsilon (10^{-16}–10^{-14}) over 100-step stateful sequences.

Timing. Wall-clock per complete simulation over 200 sequential runs of each deployed scheme on one idle core of an Apple M4 Pro laptop (native code compiled with rustc 1.99 at the release profile with link-time optimization; 64 -bit floats throughout; single-threaded). Quoted values are the median of five timed batch repeats after one warm-up; the standard deviation over the repeats is 0.3–5.4% of the median. A flown simulation spans 613–771 guidance updates on average depending on the scheme, giving the per-update and per-replan shares quoted in Section 7.2 (whole-simulation cost divided by update count – a loose upper bound on pure guidance inference). These are implementation-and-host measurements supporting the relative comparison; flight-processor worst-case execution time and memory footprint are not established here (Section 9).

Artifacts. The simulator, training harness, analysis code, every configuration, the deployed network weights, the committed per-run evaluation records behind each table, and the scripts that regenerate every figure and number are released publicly under an MIT license at https://github.com/GovitPowerz/Aerocapture. Every number in the tables regenerates, without retraining, from those records.

Appendix B: architecture probes – CfC, xLSTM, and the Mamba-3 axes

A controlled negative result supporting the Section 6 headline: does any recent recurrent family beat the plain selective SSM on the sizing tail? Three probes, each anchored on a Section 6 cell at matched parameter budget: a closed-form continuous-time cell (CfC (Hasani *et al.*, 2022), hypothesis: input-dependent time constants suit the fast-near-periapsis, static-in-vacuum phase structure) against the GRU anchor; the exponential-gated sLSTM and matrix-memory mLSTM of xLSTM (Beck *et al.*, 2024) (hypothesis: sharp revision of a stored estimate at the bounce or a density shock) against the LSTM anchor; and a 2×2 over Mamba-3's (Lahoti *et al.*, 2026) two axes – exponential-trapezoidal discretization and complex (rotational) state – at the deployed Mamba anchor, whose Euler-real arm is bit-identical to the deployed Mamba layer.

This appendix is a development-regime study (Section 2.2): every arm and its baseline trained and scored under the shared density history, so its verdicts are within-regime comparisons. Every arm shares one training protocol (the genetic algorithm at population 300 for 5000 generations, two scenarios per individual, adaptive hardest-seed curation, live actuator scaffolding) and one reserved evaluation pool, the probe pool (offset 10^7, $n = 1000$; each arm scored with its co-trained scaffolding), with three seed-repeats per arm; σ_{run} is the standard deviation over repeats. Capture is 99.97–100% everywhere and every arm passes the feasibility check of Section 6.2 – zero heat-flux, g-load, and heat-load violations on every repeat – so the comparison is a pure tail-Δv story. Two caveats apply throughout: these are p_{95}/CVaR_{95} statistics at $n = 1000$, not the far-tail $\text{CVaR}_{99.9}$ sizing metric, and the probe budget is deliberately sub-headline (1.5M evaluations versus the deployed $512 \times 20,000$), so the absolute values sit above the headline numbers and are not mission figures.

Table 11: The nine probe arms: three-seed means, correction Δv in m/s on the shared $n = 1000$ probe pool. Baselines are retrained in-regime (not the higher-budget reference runs – see the budget caveat below). Viol. is the any-constraint violation rate; every arm is feasible on every repeat.

Arm	Params	Capture %	Viol. %	p_{50}	$p_{95} \pm \sigma_{\text{run}}$	$\text{CVaR}_{95} \pm \sigma_{\text{run}}$
Mamba (baseline)	962	99.97	0.0	114.0	121.6 ± 0.5	124.1 ± 0.3
Mamba-3 trapezoidal	978	100.0	0.0	115.5	124.9 ± 2.1	128.8 ± 2.3
Mamba-3 complex	1154	100.0	0.0	113.7	121.1 ± 1.8	123.8 ± 2.1
Mamba-3 both	1170	100.0	0.0	114.0	121.6 ± 1.3	124.2 ± 1.2
GRU (baseline)	1014	100.0	0.0	114.7	123.7 ± 1.5	126.7 ± 1.3
CfC	1003	100.0	0.0	116.5	126.5 ± 0.7	130.4 ± 0.1
LSTM (baseline)	1082	100.0	0.0	115.4	124.3 ± 1.4	127.3 ± 1.4
sLSTM	1042	100.0	0.0	115.7	124.7 ± 2.6	127.6 ± 3.1
mLSTM	1078	100.0	0.0	118.1	127.4 ± 3.5	130.8 ± 4.8

Table 12: Within-family significance: gap = treatment minus baseline (positive = worse), cleared when $|\text{gap}| > \sigma_{\text{comb}} = \sqrt{\sigma_{\text{base}}^2 + \sigma_{\text{arm}}^2}$, with the per-arm σ_{run} taken from Table 11.

Treatment vs baseline	Metric	Gap	σ_{comb}	Verdict
CfC vs GRU	p_{95} / CVaR_{95}	$+2.8$ / $+3.7$	1.6 / 1.3	**significantly worse**
Trapezoidal vs Mamba	p_{95} / CVaR_{95}	$+3.3$ / $+4.8$	2.2 / 2.3	**significantly worse**
Complex vs Mamba	p_{95} / CVaR_{95}	-0.5 / -0.3	1.9 / 2.1	within run variance
Both vs Mamba	p_{95} / CVaR_{95}	$+0.0$ / $+0.1$	1.4 / 1.2	within run variance
sLSTM vs LSTM	p_{95} / CVaR_{95}	$+0.4$ / $+0.3$	2.9 / 3.4	within run variance
mLSTM vs LSTM	p_{95} / CVaR_{95}	$+3.1$ / $+3.5$	3.8 / 5.0	within run variance (high σ_{run})

The within-family rows are the rigorous claims. The CfC is significantly worse than the GRU on both tail statistics, and its tiny σ_{run} (0.1 on CVaR_{95}) makes the loss stable and reproducible, not a bad-luck draw: continuous-time time constants add a harder optimization landscape for a timescale adaptation the fixed-cadence gates already learn. Mamba-3′s trapezoidal discretization is significantly worse than the plain cell – the seed-repeats upgrade an earlier single-run "no benefit" to a measured degradation. Complex state, both axes combined, the sLSTM, and the mLSTM all sit within run variance of their baselines; the mLSTM is notably high-variance (σ_{run} up to 4.8) and trends worse without clearing the bar – matrix memory buys instability, not tail robustness. Cross-family, all nine arms share the regime and pool but the anchors differ slightly (962/1014/1082 parameters), so the ranking is suggestive rather than matched: the plain Mamba tops the field (tied with its own complex arms), about 2 m/s ahead of the GRU and 2.7 ahead of the LSTM at p_{95} – consistent with the deployed headline.

One caveat is load-bearing. Each probe also scored a higher-budget reference row: the Section 6 sweep cell for the GRU and the LSTM (population 512 for the same 5000 generations, 1.7 × the probe evaluations) and a full-budget plain-Mamba run ($512 \times 10,000$, 3.4 ×) for the Mamba. The Mamba and LSTM references sit 4–5 m/s better at p_{95} than their retrained in-regime baselines (Mamba 121.6 versus 116.6; LSTM 124.3 versus 120.2), a training-budget effect, not architecture. The GRU sweep cell sits 1.6 m/s worse than its baseline (123.7 ± 1.5 versus 125.4), so one larger-budget run can land on either side. Every treatment therefore compares against its retrained in-regime baseline and never against a higher-budget reference. Two smaller caveats: the probe cells are trained through the gradient-free path only (no warm-start), and the sLSTM's 40-parameter deficit against the LSTM is a cell-definition cost (single bias), not a budget mismatch – it does not explain its null.

The consistent null across three independent families points at the task, not the cells. A single atmospheric pass is a few hundred guidance ticks whose latent state worth remembering is a handful of slowly-varying dispersion parameters – density bias, the Ornstein–Uhlenbeck perturbation state, aerodynamic dispersions – and the engineered inputs already carry most of the temporal signal (Section 8). A diagonal selective-SSM state of dimension 12–16 saturates what little internal memory the problem rewards; the CfC's timescale adaptation, the xLSTM's revision and associative recall, and Mamba-3′s long-context and state-tracking axes all target capacity this smooth, low-bandwidth control signal never exercises. The deployed cell wins not by more sophisticated memory but by just enough memory, cheaply. Framed positively, the probes validate the methodology: the adaptive-seed, tailled, matched-anchor protocol distinguishes between architectures rather than rubber-stamping the newest one – three 2024–2026 recurrent families tried, none better, and two arms (the CfC and the trapezoidal discretization) significantly worse.

Appendix C: quantizing the deployed Mamba head

Can the deployed 962-parameter policy shrink for flight memory without giving back the tail it was selected for? This appendix quantizes the deployed head: weight-only, symmetric, fake-quantized. Rounded values are stored back as 64-bit floats, so the runtime, the regression goldens, and the simulation are untouched by construction. A post-training quantization (PTQ) grid crosses bit width (8/6/4/3/2), scale granularity (per-channel, per-tensor), and tensor policy (all tensors, or projections only, keeping the SSM dynamics and bias scalars in floating point); two quantization-aware training (QAT) arms then retrain at the best 4-bit cell of the grid. Selection ran on the fresh re-quote pool of Appendix A ($n = 1000$ per grid cell, per-scenario noise); the finalists below are quoted once on the frozen confirmatory pool of Section 7 ($10 \times 100,000$ scenarios each), paired by replicate against the deployed policy, preserving the paper's selection-versus-quote discipline.

The grid's shape is simple (Figure 19). Eight-bit quantization is free: every 8-bit cell captures every scenario within 0.5 m/s of full precision at CVaR_{95}, with every tensor quantized or not. Degradation is visible from 6 bits and catastrophic at 2. At 4 bits – the target compression – the selection rule (highest capture rate, then lowest CVaR_{95}) picks per-tensor scales on the projection tensors only: it is the one 4-bit cell at full capture (CVaR_{95} 222 m/s against 123 in full precision), where per-tensor all-tensors captures 99.4% and the two per-channel cells 91.6% and 64.9%; at 3 bits only the same cell stays near full capture (99.2%). The ranking between granularities is a rounding draw rather than a granularity effect: per-channel scales have the lower weight error, and a jitter sweep in the repository that enlarges every quantization step by a few percent moves the 4-bit cells across one another. The policy split is the robust finding; the granularity verdict is one draw.

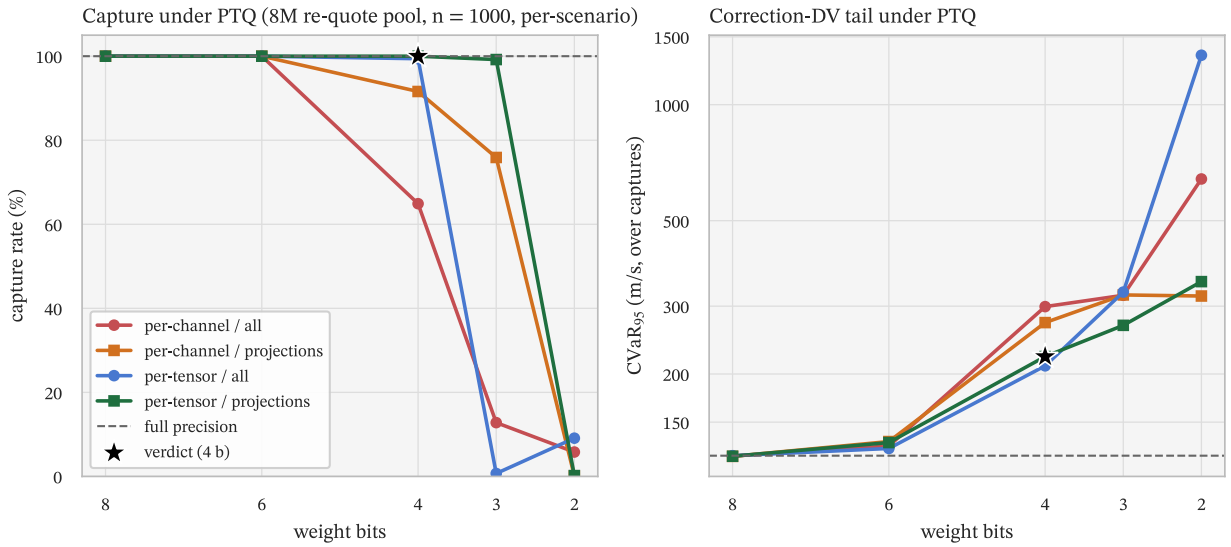


Figure 19: The post-training quantization grid on the re-quote pool ($n = 1000$ per cell, per-scenario noise): capture rate (left) and CVaR_{95} on a log axis (right) versus bit width, for the four granularity $\times$ policy series; the verdict cell is starred.

Table 13: Quantization finalists: capture, CVaR_{95}, $\mathrm{CVaR}_{99.9}$ and the count of heat-load exceedances on the frozen confirmatory pool ($10 \times 100,000$ scenarios each, per-scenario noise; correction Δv in m/s; replicate standard errors), p_{50} on the re-quote pool ($n = 10,000$), and the paired replicate delta of $\mathrm{CVaR}_{99.9}$ against the deployed policy with its 95% interval over the ten replicates. The 4-bit cells use per-tensor scales on the projection tensors only. No cell exceeds the heat-flux or g-load limits beyond one scenario.

Variant	Capture %	p_{50}	CVaR_{95}	$\mathrm{CVaR}_{99.9} \pm \mathrm{SE}$	$\Delta\mathrm{CVaR}_{99.9}$ paired	Heat load
Deployed (64-bit)	100.0000	110.2	120.1	173.9 ± 2.2	–	1148
PTQ, best 4-bit	99.9702	164.3	220.2	286.5 ± 0.7	$+112.7$ [+109.2, +116.3]	97
QAT fine-tune, 4-bit	100.0000	111.0	123.8	153.7 ± 0.7	-20.0 [−24.3, −15.7]	1019
QAT scratch, 4-bit	100.0000	111.6	122.9	175.5 ± 2.7	$+1.7$ [−5.2, +8.6]	505

Post-training rounding is not deployable: the best PTQ cell costs $+100$ m/s at CVaR_{95} and $+113$ at $\mathrm{CVaR}_{99.9}$, and loses 298 of 10^6 scenarios. Quantization-aware training recovers the tail by either route. The champion checkpoint resumed for 3000 generations with every candidate's weights fake-quantized before each fitness evaluation, so the genetic search optimizes the quantized policy directly, pays $+3.8$ m/s at CVaR_{95} and gains 20.0 at $\mathrm{CVaR}_{99.9}$ (interval excluding zero) at full capture; the same 4-bit constraint trained from scratch at the champion's full $512 \times 20,000$ budget matches the champion's far tail ($+1.7$, interval across zero) at $+2.8$ m/s of CVaR_{95}. Both intervals condition on this champion, the worst far tail of its three seeds (Section 6.2); the fine-tune row sits inside that family's range and the scratch row 1.5 m/s above its worst seed: a 4-bit projections-only head is tail-equivalent to the full-precision family at sizing depth, by fine-tune or from scratch, and not better than it. The fine-tune's deployed head is the validation gate's last promotion, and its final population was wholly gate-infeasible on heat load – the gate kept the feasible head.

Which parts of a selective SSM tolerate 4 bits? A one-tensor-at-a-time probe quantizes a single tensor group at 4 bits per-tensor and keeps everything else in floating point (re-quote pool, $n = 1000$ per cell):

Table 14: One-tensor-at-a-time sensitivity at 4 bits per-tensor ($n = 1000$ per cell, re-quote pool, per-scenario noise). The SSM input projection and the input dense layer are the sensitive groups on this champion. The deltas do not add up to the all-tensors cell (Δv statistics are conditional on capture) – quantization errors interact.

Tensor group	Params	Capture %	$\Delta\mathrm{CVaR}_{95}$
input dense W	272	100.0	$+51.6$
SSM input projection $W_{x,\mathrm{proj}}$	400	100.0	$+107.6$
SSM Δ-projection W_{Δ}	16	100.0	$+10.9$
SSM dynamics $a_{\log}$	192	100.0	$+5.3$
SSM residual gains d_{skip}	16	100.0	$+18.7$
output dense W	32	100.0	$+0.0$

The SSM input projection is the bottleneck on this champion ($+108$ m/s from one tensor), with the input dense layer next; the dynamics parameters $a_{\log}$ and the residual gains, which an earlier champion trained on the shared density path had put first, cost $+5$ and $+19$ here. The sensitivity map is a property of the trained weights, not of the architecture, which is why the policy split (quantize the 720 projection weights, keep the 242 dynamics and bias scalars in floating point) is selected on the grid per champion rather than assumed. The Δ-projection costs $+11$ under one per-tensor scale (per-channel scaling of its single column would be lossless by construction).

The deployment benefit, stated plainly, is memory – not compute:

Table 15: Analytic memory footprint (packed b-bit weights + 32-bit scales + floating-point remainder; 64-bit baseline 7696 B). The all-tensors cell is shown for contrast only – it loses capture at 4 bits. The headline is the deployed cell, bounded below by the 968 B of dynamics and bias parameters kept in floating point.

Cell	Packed	Scales	Float	Total	vs 64-bit
8-bit per-channel, projections	720 B	236 B	968 B	1924 B	$4.0\times$
4-bit per-channel, projections	360 B	236 B	968 B	1564 B	$4.9\times$
4-bit per-tensor, projections (deployed QAT cell)	360 B	16 B	968 B	**1344 B**	$\mathbf{5.7\times}$
4-bit per-tensor, all tensors	464 B	24 B	136 B	624 B	$12.3\times$

On compute, the head accounts for ≈ 1.46 ms of the deployed policy's 3.28 ms whole-simulation cost (Section 7.2), a measurement of the architecture that the champion's weights do not change:

Table 16: Forward-pass micro-benchmark (median ns per guidance tick, laptop-class core, scalar code, the SSM recurrence in floating point for every variant; ms per simulation over the deployed seed's mean of 771 forward passes per flown trajectory, Section 7.2). Quantization is not a compute win at this scale.

Forward kernel	ns / tick	vs deployed	ms / simulation
64-bit (deployed runtime)	1888	–	1.46
64-bit hand-rolled	1670	-11.6%	1.29
32-bit hand-rolled	1290	-31.7%	0.99
int8 weights, int8 activations	1735	-8.1%	1.34
packed int4 weights, int8 activations	1747	-7.5%	1.35

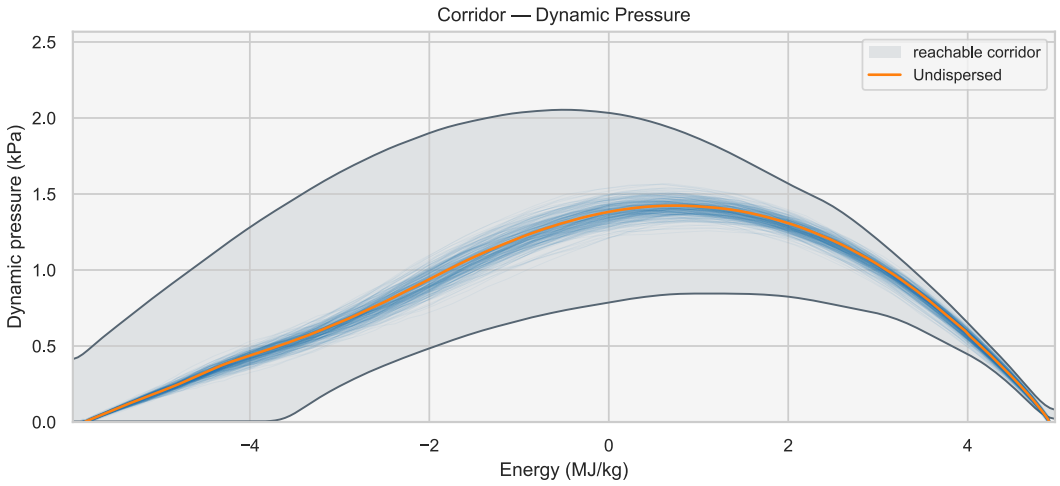
Single precision is the compute sweet spot (-32% per tick); the integer kernels beat the deployed runtime by only $\approx 8\%$, because dynamic activation quantization plus the floating-point SSM recurrence (192 exponentials per tick) dominate a 962-parameter workload, and 4-bit nibble unpacking cancels its bandwidth advantage. Simulation throughput in the accuracy study is unchanged by construction, so no simulation-time claim is made. The deployment case for the 4-bit head is the $5.7\times$ memory footprint and fixed-point-capable projection arithmetic, not speed.

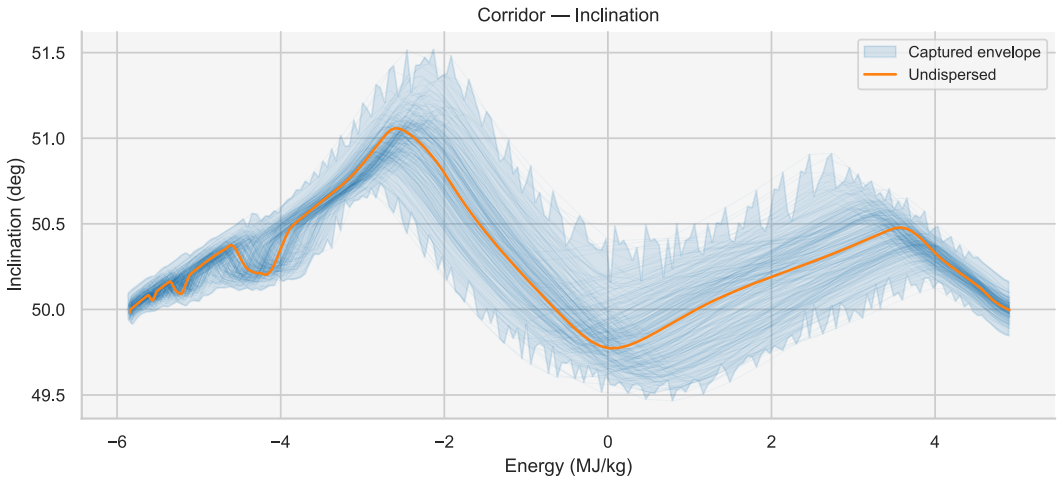
Four caveats bound these claims. The accuracy study is weight-only: activations, the hidden state, and the input normalization stay in 64-bit floats (the integer kernels quantize activations for the compute measurement only). Both QAT arms are single runs. The grid cells are single-model $n = 1000$ evaluations. And the kernel ratios are laptop-class scalar measurements that do not transfer to flight processors – the memory table does.

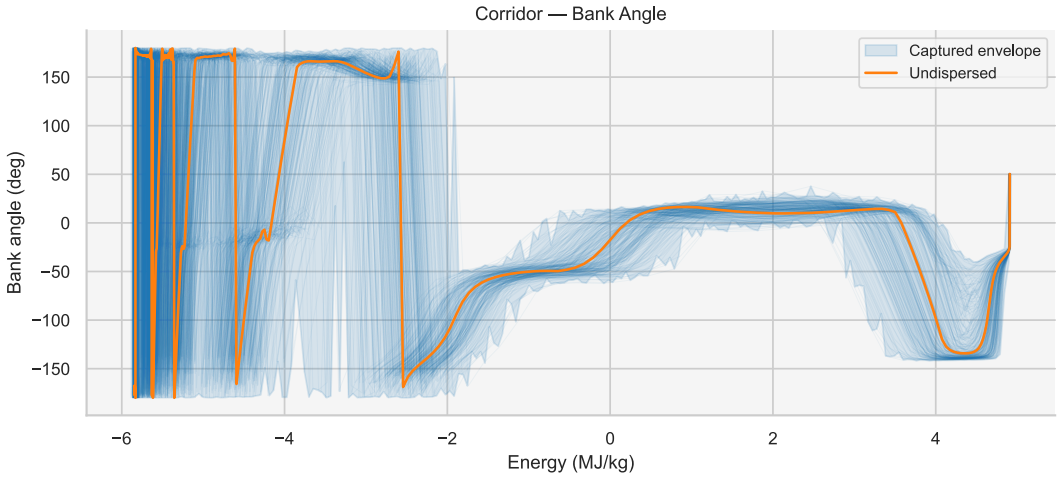
Appendix D: per-scheme mission reports

Three two-page mission-performance cards on the final-evaluation Monte Carlo pool ($n = 1000$, per-scenario noise), each pinned to its deployed policy so the statistics reproduce the corresponding row of Table 7: the deployed Mamba seed, the retuned joint-FTC (its co-optimized-reference variant, Section 7.1) and the retuned FNPAG. The first page shows the corridor behaviour – the classified trajectory ensemble in the (energy, dynamic pressure), (energy, inclination), and (energy, bank) planes, with the undispersed nominal overlaid; the dynamic-pressure panel sets the ensemble against the shared occupancy envelope of Figure 1, so one can read at a glance whether each scheme flies inside it. The second page shows the correction-Δv distribution (total and its three burns), the thermal and load-constraint margins, and the full statistics. Panels reuse the training pipeline's own report charts; captured trajectories are blue, constraint violations orange, failures red. The repository's generated reports carry the same card for every other cell of this paper.

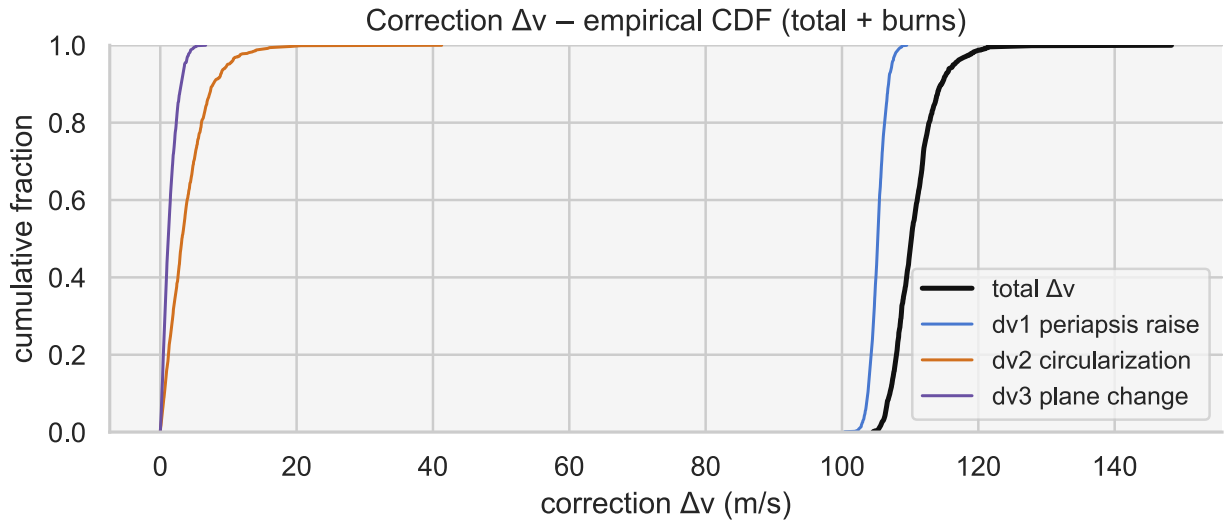
NN -- Mamba (962 params, deployed)

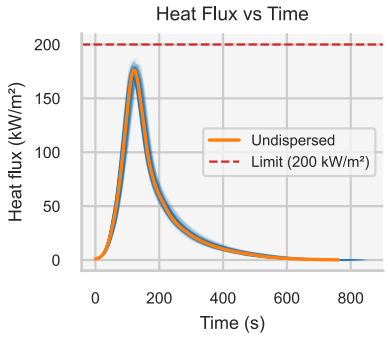


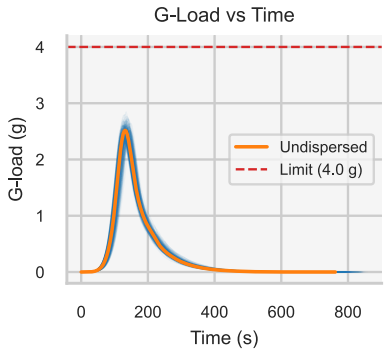


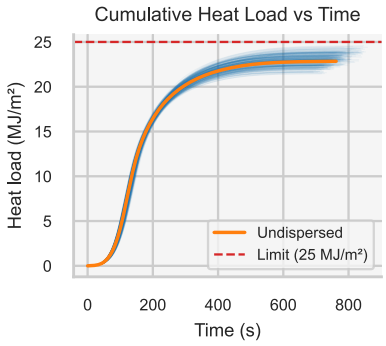


NN -- Mamba (962 params, deployed) (continued)



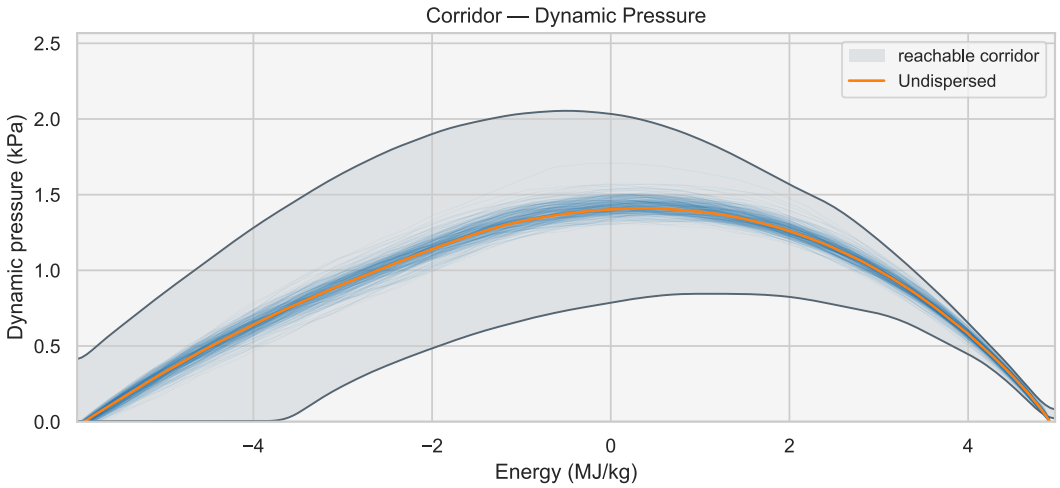


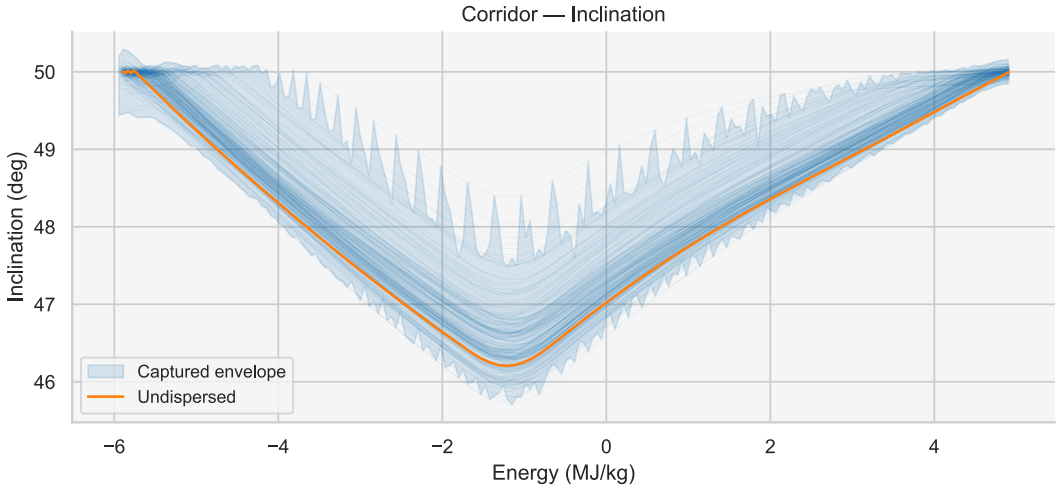


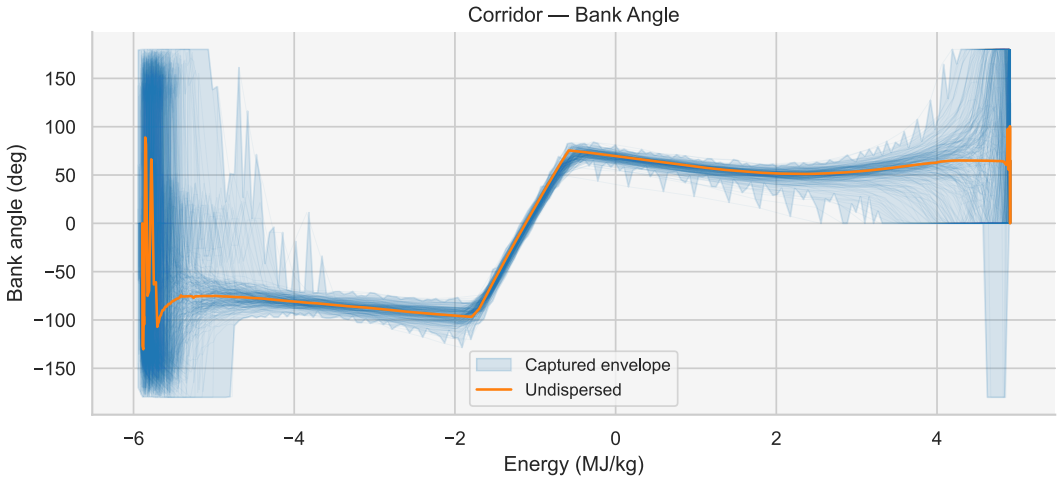


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	110.2	116.4	110.6	mean
Δv CVaR95 / p99 / max	119.7	120.7	148.3	tail
dv1 periapsis raise	105.2	107.3	109.5	m/s
dv2 circularization	3.2	9.8	41.3	m/s
dv3 plane change	1.2	3.6	6.7	m/s
Apoapsis error (km)	9.5	45.8	10.3	mean
Periapsis error (km)	30.6	37.4	30.5	mean
Inclination error (deg)	0	0.05	0	mean
Heat flux (kW/m²)	176.3	182.5	187.8	viol 0%
G-load (g)	2.52	2.69	2.86	viol 0%
Heat load (MJ/m²)	23	24.1	25	viol 0.1%
Capture rate				100%

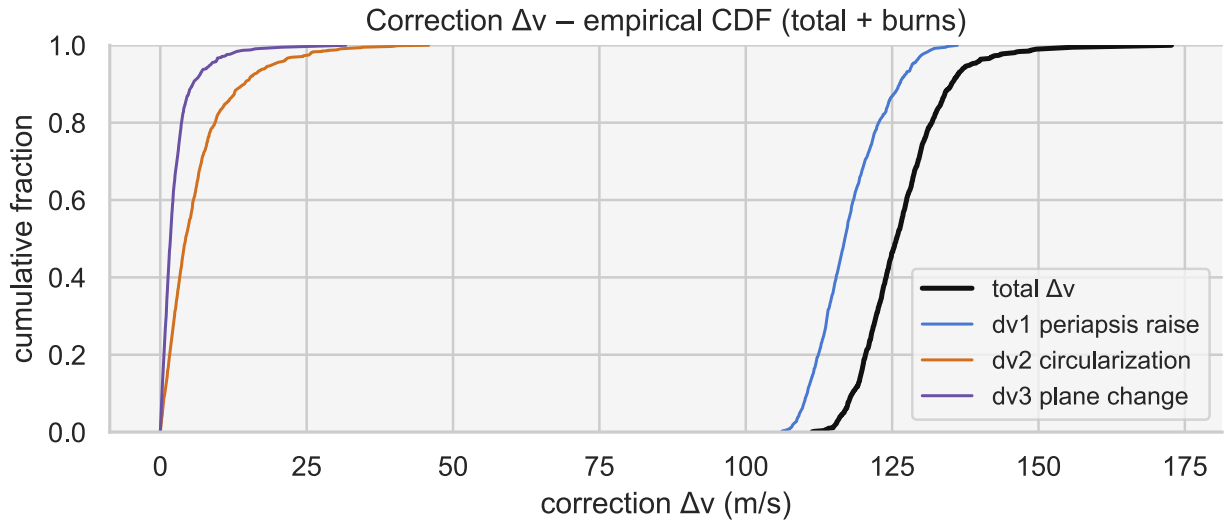
FTC (joint reference, retuned)

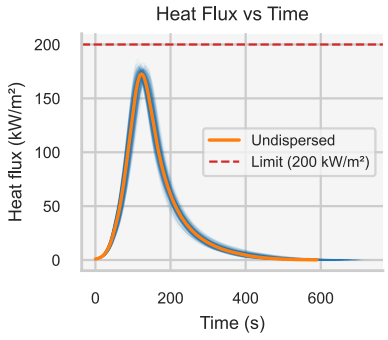


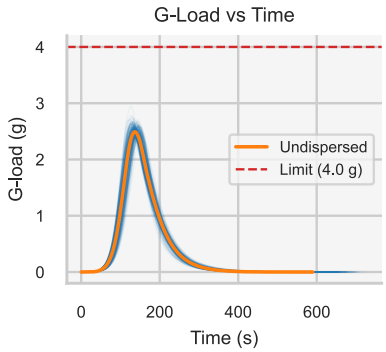


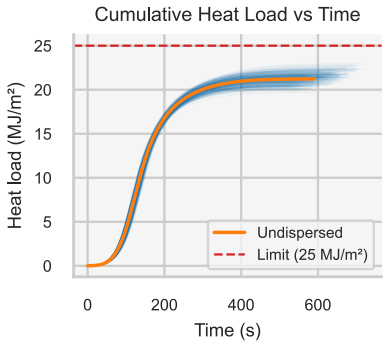


FTC (joint reference, retuned) (continued)



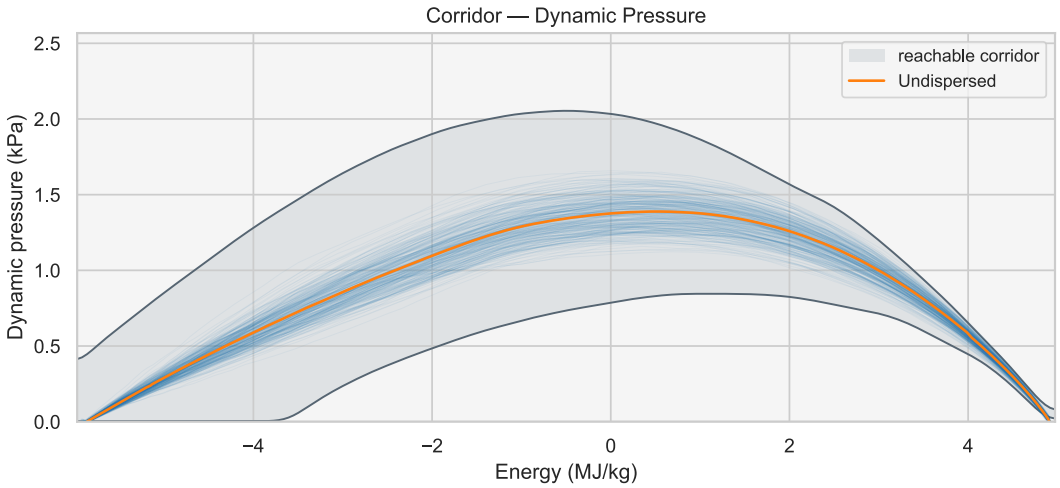


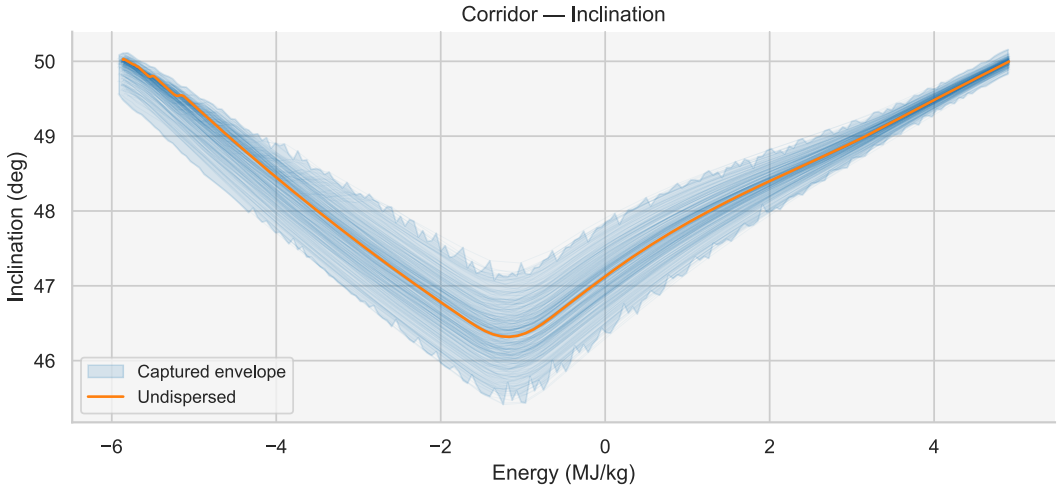


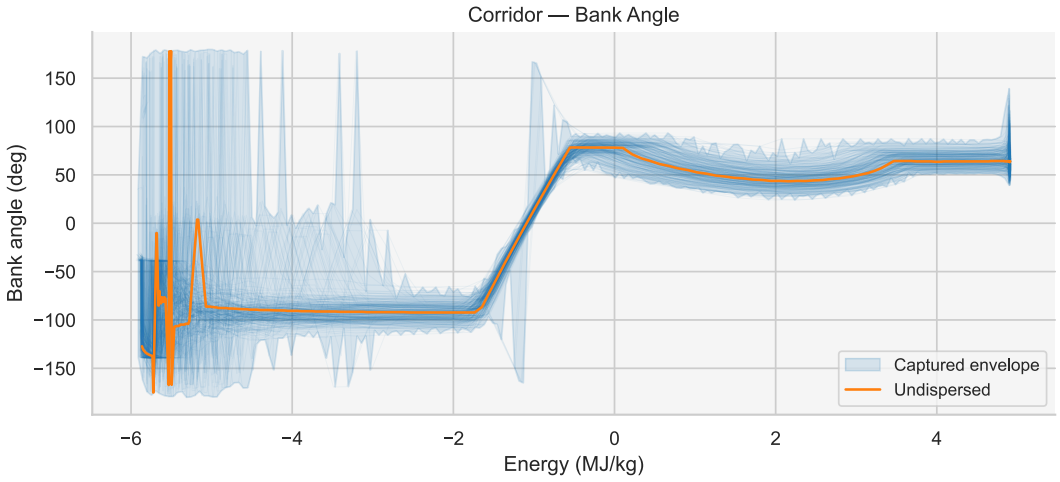


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	125.7	138.4	126.6	mean
Δv CVaR95 / p99 / max	146.3	149.6	172.7	tail
dv1 periapsis raise	116.9	128.4	136.1	m/s
dv2 circularization	4.3	19.2	45.8	m/s
dv3 plane change	1.7	8.6	31.7	m/s
Apoapsis error (km)	0.5	91.5	8	mean
Periapsis error (km)	−15.3	12.3	−18.9	mean
Inclination error (deg)	−0.01	0.06	−0.02	mean
Heat flux (kW/m²)	174.2	180	186.8	viol 0%
G-load (g)	2.54	2.68	2.95	viol 0%
Heat load (MJ/m²)	21.3	22.4	23.1	viol 0%
Capture rate				100%

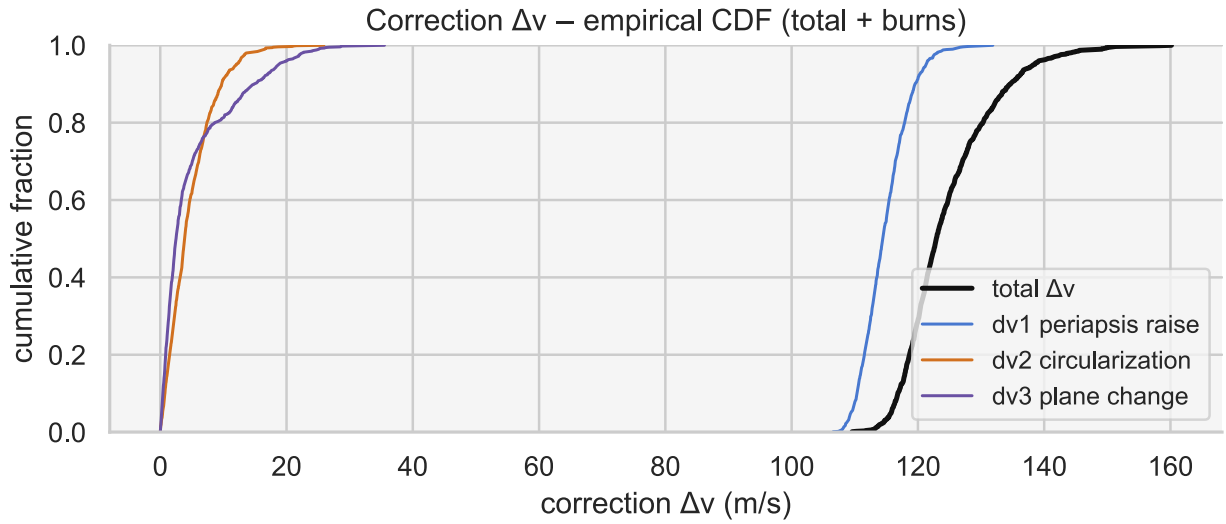
FNPAG (retuned)

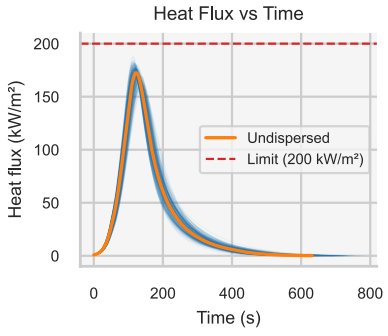


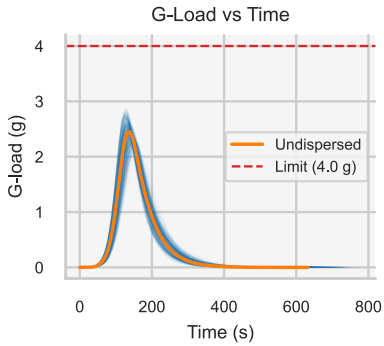


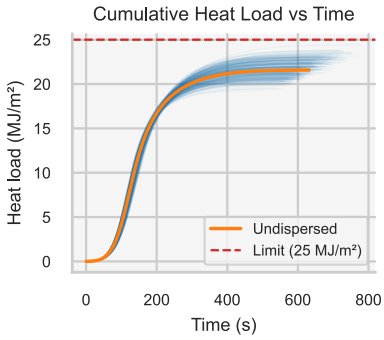


FNPAG (retuned) (continued)









Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	122.9	138.4	124.6	mean
Δv CVaR95 / p99 / max	144.2	149.2	160.2	tail
dv1 periapsis raise	114.4	121.3	131.9	m/s
dv2 circularization	3.8	12	25.9	m/s
dv3 plane change	2.5	18.7	35.5	m/s
Apoapsis error (km)	9.1	55.5	10.2	mean
Periapsis error (km)	−6.1	13.4	−7.6	mean
Inclination error (deg)	−0.03	0.06	−0.07	mean
Heat flux (kW/m²)	172.5	181.7	189.9	viol 0%
G-load (g)	2.47	2.78	3.06	viol 0%
Heat load (MJ/m²)	21.7	23.3	24.8	viol 0%
Capture rate				100%

Provenance. Every cell of Table 4, Table 5, Table 6, Table 7, Table 8, Table 9, Table 10, Table 13, Table 14 and Table 15, every per-scenario number of Sections 5–10 and Appendix C, the compute quotes, the heat-load ceiling sentence of Section 6.2 and the budget

caveat of Appendix B are read at compile time from `data/results.json`, `data/confirmatory_marginal.json`, `data/quote_marginal.json`, `data/heat_load_slope.json`, `data/stress_depth.json`, `data/centered_depth_v4.json`, `data/objective_centering_v4.json`, `data/quant_v4/`, `data/compute_benchmark.json`, `data/probes/` and the deployed seed's bundled run directory through `results.typ`, and the counts and orderings the prose states are asserted against the same files. Sections 4–5, the Section 6.1 sweep, the bank-decoder comparison and Appendix B quote development-regime numbers under that label; Appendix C's forward-kernel micro-benchmark is transcribed. Paper inputs digest (SHA-256 over every tracked paper input, `data/provenance.json`): `83afb41a5d9cfdca85e3dfebb93fbb25faf1f63146cc6edd90f3e7db1f595b99`. Run logs: Release `arxiv-v4`. Simulator crate 1.0.0, Typst 0.15.1, matplotlib 3.11.2. Noise regime: per_draw (one density-noise realization per scenario, ADR-0006) for the main body: every rl/* and ou_marginal/* run of results.json (issues #101, #175), every cell of confirmatory_marginal.json (issues #137, #174), stress_depth.json (issue #176), objective_centering_v4.json and centered_depth_v4.json (issue #177), quant_v4/ (issue #178); per run in results.json (ADR-0003). The shared-path (legacy) files are the development regime: every other results.json run, confirmatory_eval.json, far_tail_eval.json, robustness_*.json, objective_centering.json, quant/. quote_marginal.json pins legacy for both of its regimes and re-seeds simulation.random_seed per scenario for the marginal one (issue #157), the regime heat_load_slope.json scores under (issue #192); centered_depth.json carries both regimes, each cell labelled (issue #156). Compiled at git `c4187888`.